Ariadne
3D Dungeon Maker

Manual

Ver 1.5.1

Chapter 1

Overview

Ariadne 3D Dungeon Maker is a powerful asset to create 3D dungeons. In the extension, you can make grid-based map data in MapEditor and set it to the Game Controller prefab. Ariadne 3D Dungeon Maker produces dungeons according to your map data at runtime.

This asset includes the MoveController that enables movement in the dungeon.

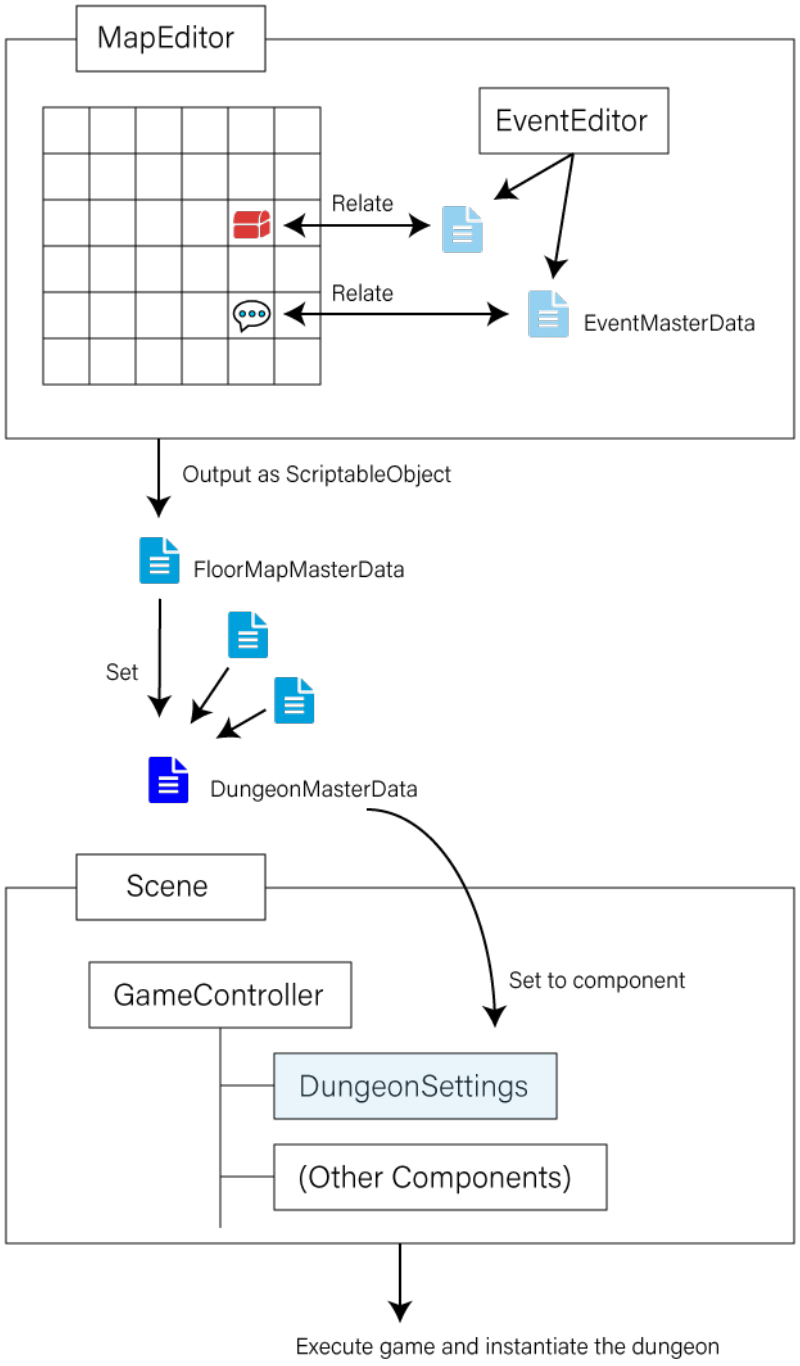
The controller enables processing events, too.

Chapter 2

Getting Started

Data flow

Here is the data flow of Ariadne 3D Dungeon Maker.



Workflow to Generate Dungeon

Create a map data by using MapEditor

First, create map data by using MapEditor. You can open the MapEditor from **[Window] -> [MapEditor]** in the menu bar.

On the MapEditor, you can set attributes of the map by using draw tools.

To add an event, select a position using the select tool and assign event data at the position. You can create new event data, and you can also assign existing data. After assigning an event data, press **[Open Event Editor]** button.

On the EventEditor, you can define the contents of events and starting conditions.

Set the map data to Dungeon Data

After saving the map data, create a dungeon data from **[Create] -> [Ariadne] -> [DungeonData]** in the context menu.

The dungeon data is a holder of map data. Set map data that you created to dungeon data.

Ready objects in the scene

Next, set the dungeon data to GameController object in the Scene. The GameController object has a component named DungeonSettings, so set the dungeon data to this component.

Template objects are placed in **[Ariadne/Prefabs/SceneObjects/Template]** folder as prefabs. When you intend to create a new scene, it is useful to duplicate the demo scene and customize it or instantiate template prefabs.

Execute the game

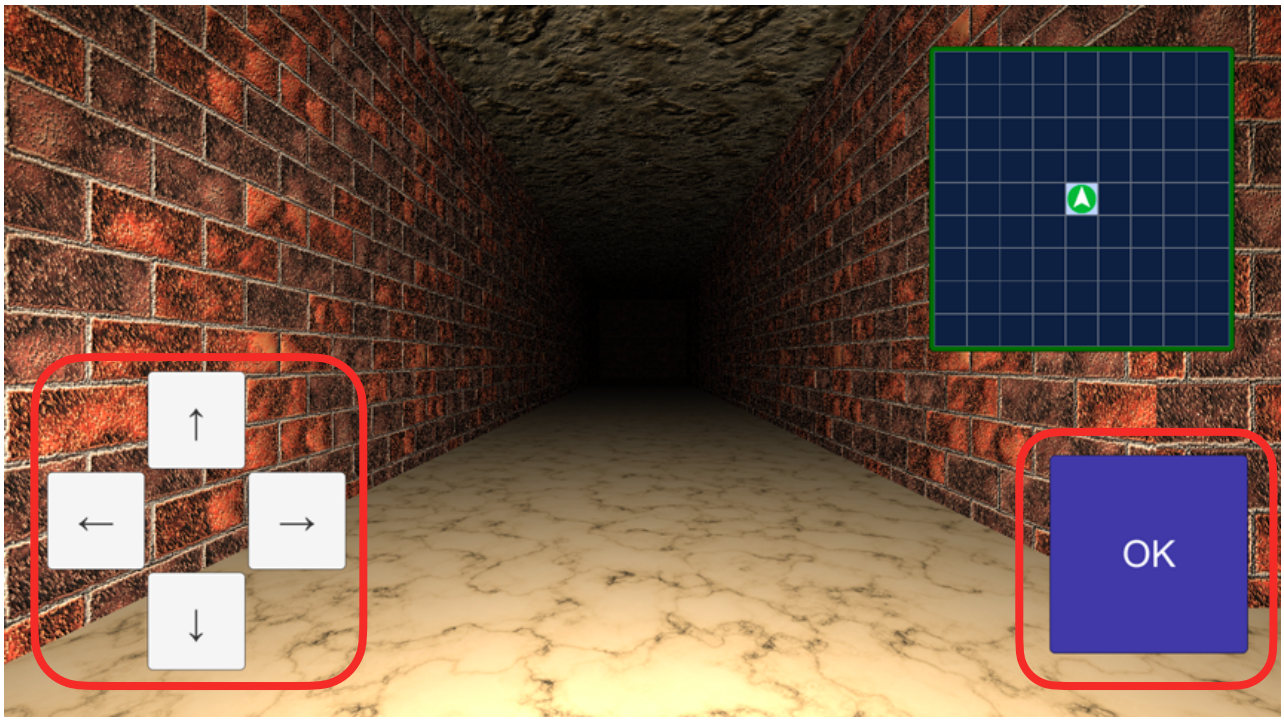
Now, ready to explore your dungeon. Execute the game!

Operation in the Dungeon

To move in the dungeon, you can use arrow keys and space key.

Key	Action
Up Arrow	Moves the player to forward.
Left Arrow	Turns the player in a counterclockwise direction. When pressing the shift key, the player moves to left without changing direction.
Right Arrow	Turns the player in a clockwise direction. When pressing the shift key, the player moves to right without changing direction.
Down Arrow	Turns the player around to back. When pressing the shift key, the player moves to back without changing direction.
Space	Start the event and decide.

And you can also use uGUI buttons on the screen. These buttons can be set invisible in the MoveController component.



Corresponds to arrow keys

Corresponds to space key

Chapter 3

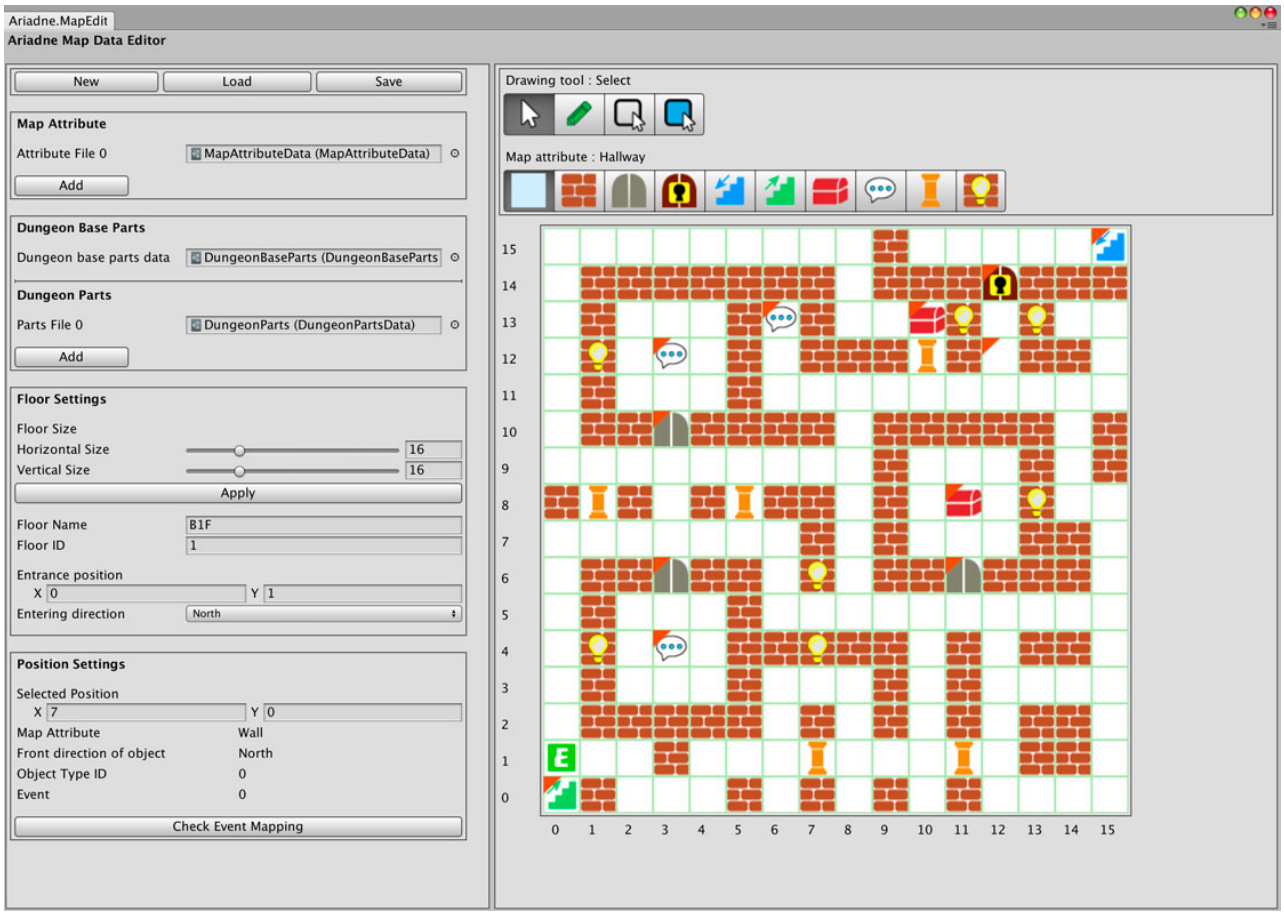
System windows in Ariadne

This chapter explains system windows and editors which edit data used in Ariadne.

Ariadne has the following windows/editors.

Name of Window/Editor	Related Data	Description
MapEditor	FloorMapMasterData	You can create and edit floor data of dungeons in MapEditor. Set map attributes and events to the position on the floor.
EventEditor	EventMasterData	You can edit events in the dungeon by using EventEditor.
EventMappingViewer	EventMasterData	EventMappingViewer show relationships between FloorMapMasterData and EventMasterData.
EventCategoryEditor	EventCategoryData	EventCategoryEditor defines categories of events. These categories are combined in event processes.
EventArgumentTypeEditor	EventArgumentTypeData	EventArgumentTypeEditor defines argument types that are used in event category data.
MapAttributeEditor	MapAttributeData	MapAttributeEditor defines map attributes in the dungeon.
DungeonPartsEditor	DungeonPartsData	DungeonPartsEditor defines parts of dungeons (such as wall objects, door objects, and so on) which corresponds to map attributes.

MapEditor

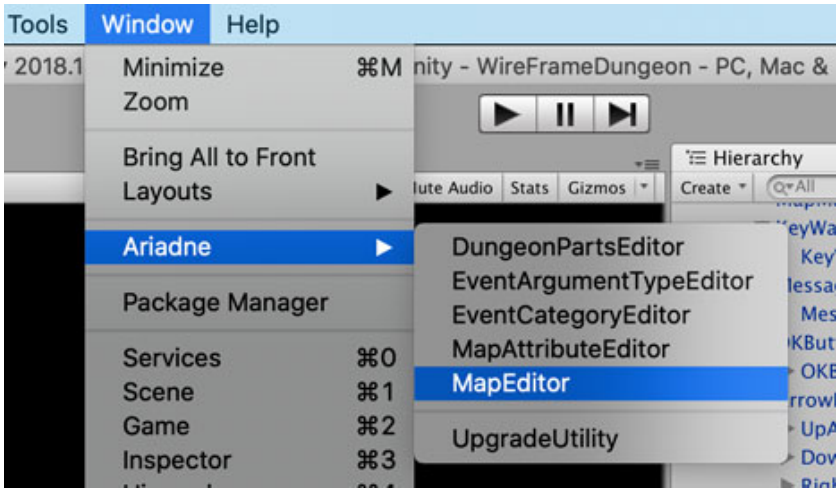


MapEditor is a core function of Ariadne 3D Dungeon Maker. MapEditor edits and outputs **FloorMapMasterData** which defines the size of the floor, map ID, map name, and detailed information about each position in the map.

See also the description of FloorMapMasterData.

How to open MapEditor

Open MapEditor from **[Window] -> [Ariadne] -> [MapEditor]** in the menu bar.

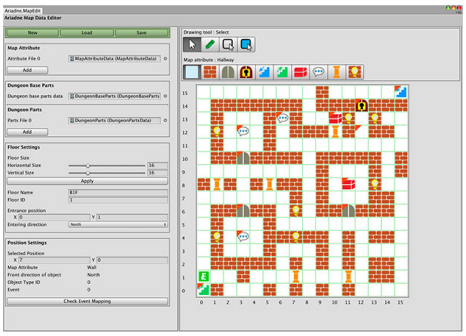


Settings pane

This section explains setting panes in MapEditor.

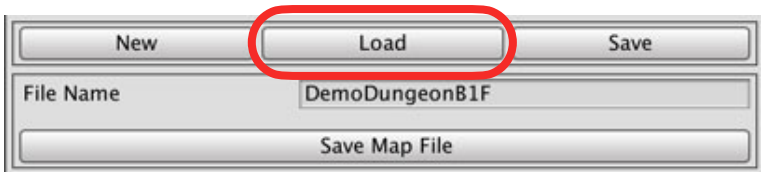
File operation pane

You can operate a FloorMapMasterData file in this pane.

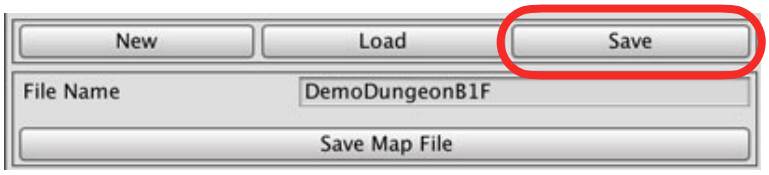


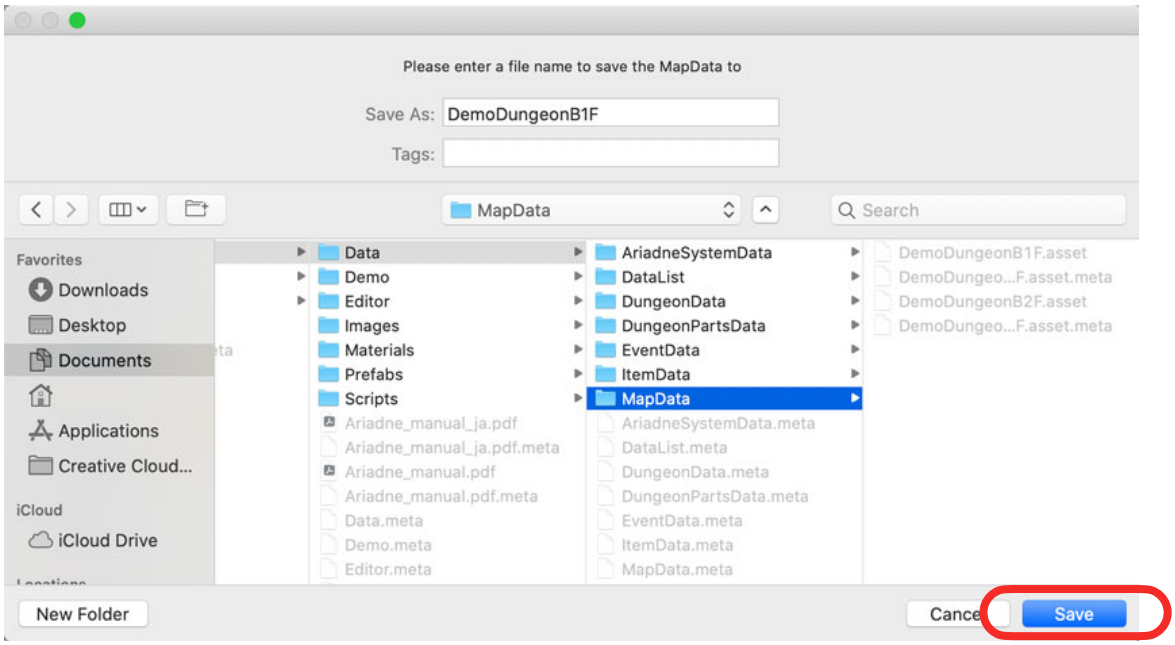
To create a new FloorMapMasterData, press **[New]** button.

[Load] button shows the load pane. On the load pane, select FloorMapMasterData to load. And press **[Load Map File]**, MapEditor loads the specified file.



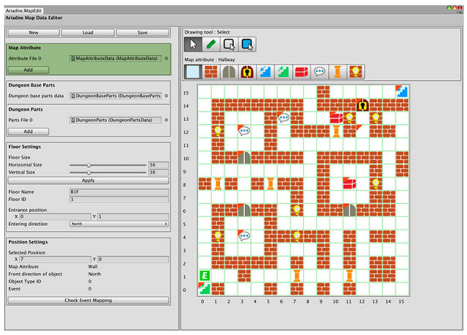
[Save] button shows the save pane. On the Save pane, specify the file name to save. And press **[Save Map File]**, the dialog will appear. Click **[Save]** to save FloorMapMasterData.





Map Attribute pane

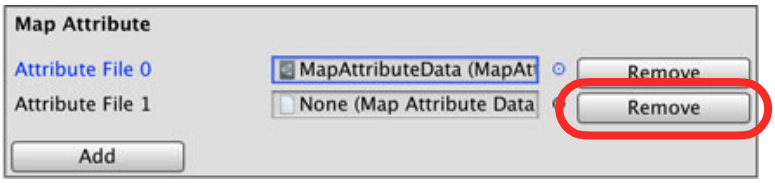
Set MapAttributeData to the editing floor.



To edit map attributes on this floor, set some MapAttributeData to **[Attribute File]** fields. When you create a new file, MapEditor set a default file to this field automatically.

[Add] button adds a new field to set MapAttributeData file. You can set multiple MapAttributeData files to a FloorMapMasterData. So you can combine system data and user-defined data.

When **"Map Attribute"** has some fields, MapEditor shows **[Remove]** buttons. These buttons remove a corresponding field.

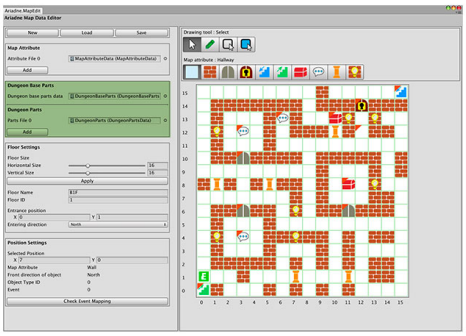


After adding MapAttributeData files, MapEditor shows new buttons that corresponding to map attributes in these files.

If map attributes in the floor are not found in these files, MapEditor doesn't show icons of map attributes on the map.

Dungeon Parts Settings pane

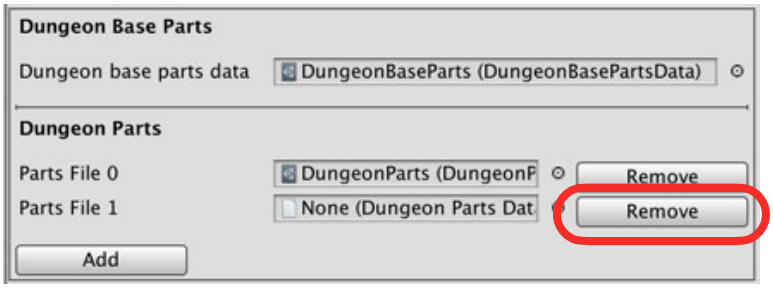
Set DungeonBasePartsData and DungeonPartsData for this floor. Ariadne uses these files to instantiate objects which corresponding to map attributes at runtime.



Set a DungeonBasePartsData to **[Dungeon base parts data]** field. DungeonBasePartsData includes the base parts to calculate the sizes of dungeon parts. It also includes references for the ceiling object, the floor object, and the outside wall object.

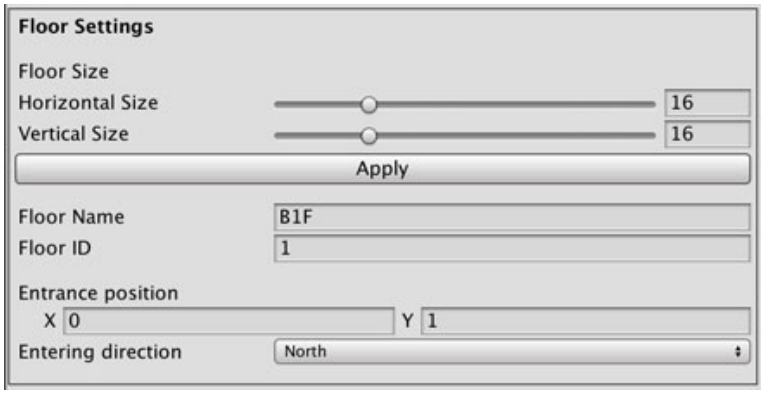
[Parts File] fields hold DungeonPartsData. If you assigned user-defined MapAttributeData, set user-defined DungeonPartsData to this field, too. **[Add]** button adds a new field to set the DungeonPartsData file.

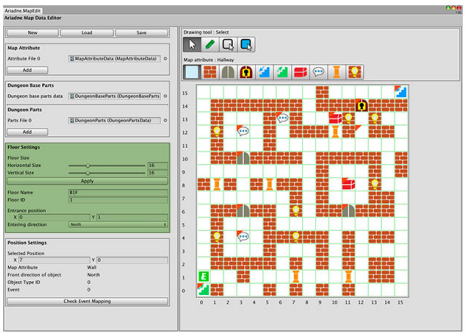
When **"Dungeon Parts"** has some fields, MapEditor shows **[Remove]** buttons. These buttons remove a corresponding field.



Floor Settings pane

Set information about the editing floor.





Define “**Horizontal Size**” and "**Vertical Size**” of the floor. To apply sizes to the map, press **[Apply]** button. If the value of the slider is smaller than the previous one, a part of the map attribute will be discarded.

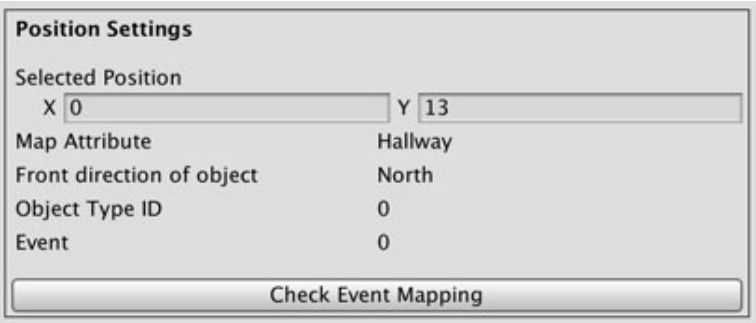
Set “**Floor Name**” and “**Floor ID**”. Floor ID is used in the DungeonMasterData to specify the start floor of the dungeon. If there are some FloorMapMasterData which have the same floor ID, the selector in DungeonMasterData shows only one of them. So you have to specify different IDs for those floors.

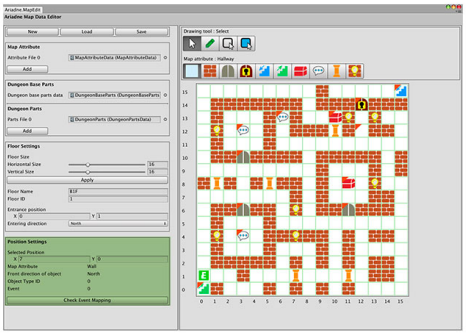
"**Entrance Position**" is the first position when the player enters this dungeon. The setting of the first floor of the dungeon is in DungeonMasterData. See also DungeonMasterData.

You can specify the direction of entering, too.

Position Settings pane

MapEditor shows information about the selected position. You can set event data at this pane.





"Position Settings" pane shows information about the selected position.

"**Selected position**" indicates the position on the grid where the mouse cursor is. When the "**Select**" tool is selected, this shows the position where you selected.

Label fields show information about the position.

"**Front direction of object**" shows the direction of dungeon parts in this position.

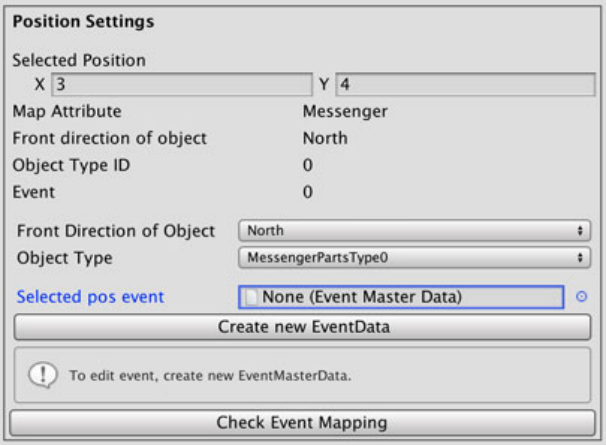
"**Object Type ID**" shows the type ID of the object which is defined in the DungeonPartsEditor.

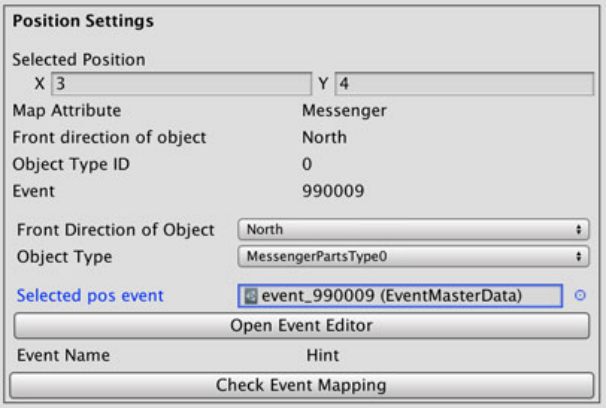
"**Event**" is the event ID of the position.

Press [**Check Event Mapping**] button shows EventMappingViewer. This viewer shows mappings between FloorMapMasterData and EventMasterData. See also EventMappingViewer.

If you select a position on the map by using the **“Select”** tool, This pane shows additional information.

The left image is the state if the **“Selected pos event”** field is null. The right image is the state if the **“Selected pos event”** field has event data.





This pane shows **“Front Direction of Object”** field if you select the position. You can set the direction of the object to instantiate.

This pane also shows **“Object Type”** field if the map attribute of this position has some object type in the DungeonPartsData. If the map attribute of this position doesn’t have an object type (such as Hallway), this field will be hidden.

If the “Selected pos event” field is null, you can set an existing event data or create new event data by clicking **[Create new EventData]** button.

When you set an existing event data, you can select data from EventDataList. If the selected event data wasn’t shown, check EventDataList file in the project.

After you set event data, you can click **[Open Event Editor]** and edit the event. See also EventEditor for more detail.

Drawing map pane

"Drawing map pane" is an editing place for attributes of each position.

Drawing tool

"**Select**" tool can pick up information about the position. You can see them in the "Position Settings pane".

"**Draw**" tool can set a map attribute value. When you drag the mouse cursor, this tool sets map attributes to positions where you drag.

"**Draw rect**" tool can set map attributes as a rectangle.

"**Draw filled rect**" tool is similar to the "Draw rect" tool. This tool fills inside of the rectangle.

Map attribute

By default, Ariadne defines follow map attributes. You can customize map attributes in MapAttributeEditor.

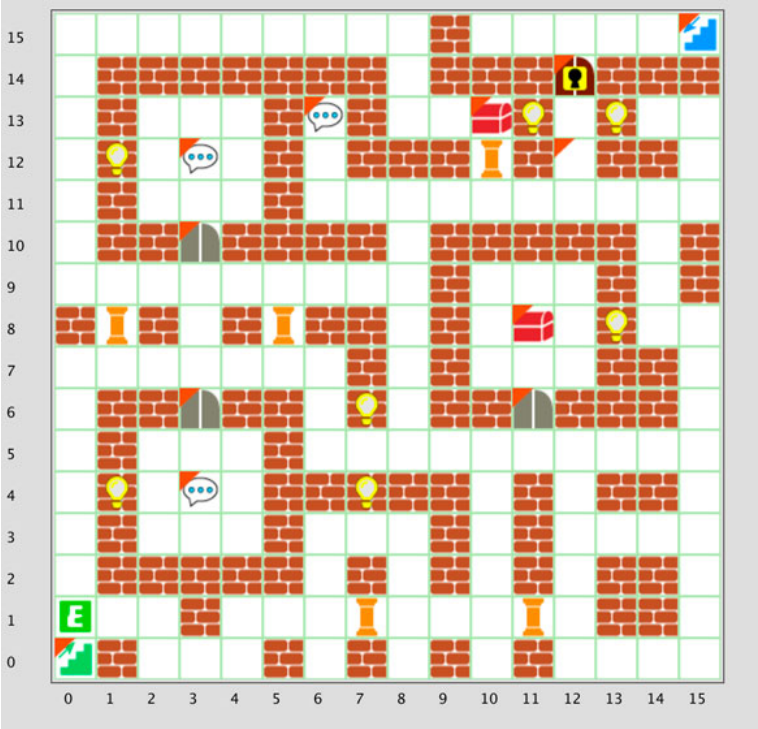
ID	Icon	Attribute	Can a player walk?
0		Hallway	Yes
1		Wall	No
2		Door	No
3		Locked Door	No
4		Downstairs	No
5		Upstairs	No
6		Treasure	No
7		Messenger	No
8		Pillar	No

ID	Icon	Attribute	Can a player walk?
9		WallWithTorch	No

Grid-based map

Ariadne 3D Dungeon Maker uses grid-based maps for instantiating dungeons and holding the player position.

The origin is left-bottom.



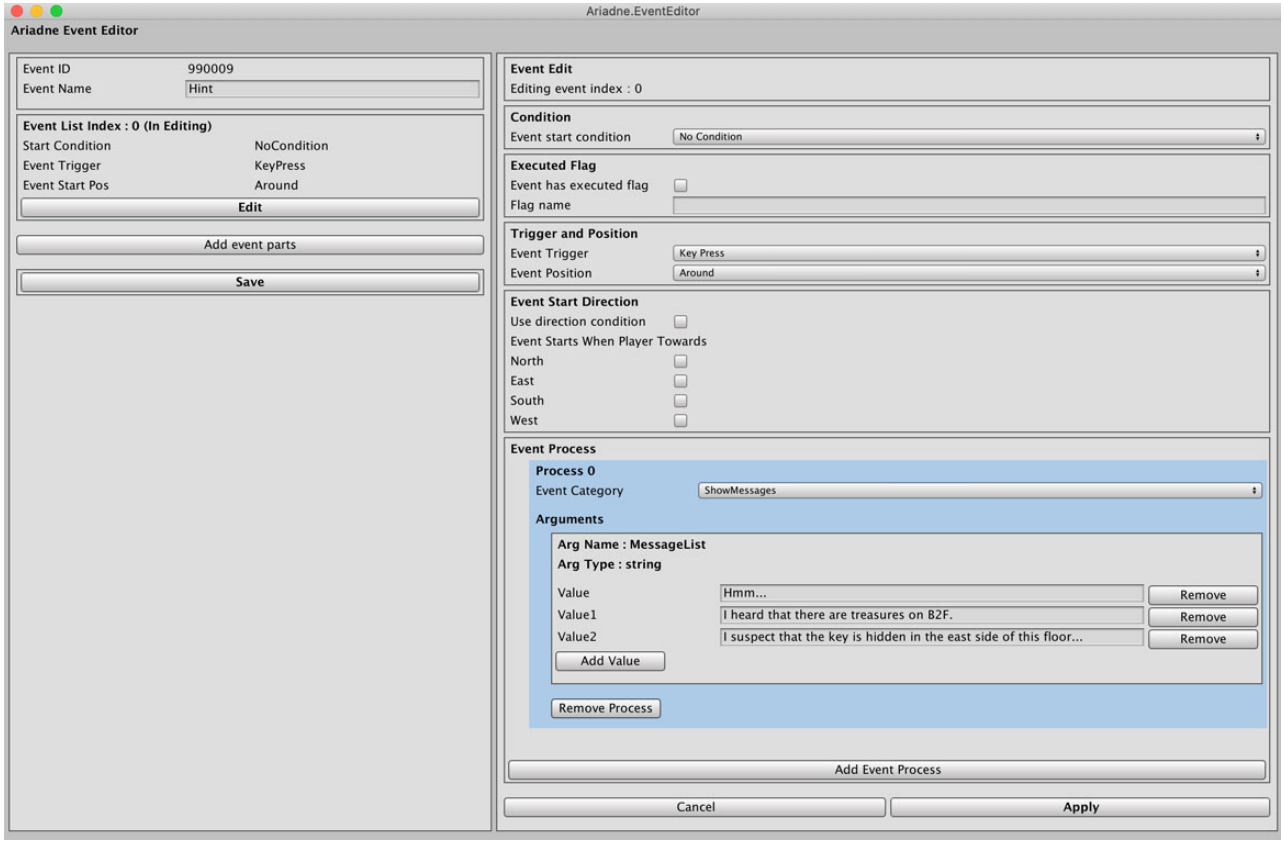
How to close the window

To close this window, click the close button on the window.

When this window lost focus, editing data will be saved as temporary data in [**Assets/Ariadne/Editor/MapEditorTemp**] folder.

Attention: Don't set temporary data to DungeonMasterData to prevent unexpected map changes. When you use FloorMapMasterData, save the file by using [**Save Map File**] button.

EventEditor



EventEditor is a tool for editing events. The event is an action on the dungeon, such as opening the door. EventEditor saves events as **EventMasterData** that inherit ScriptableObject.

How to open EventEditor

Open EventEditor by pressing [**Open Event Editor**] button on the MapEditor window.

About EventParts

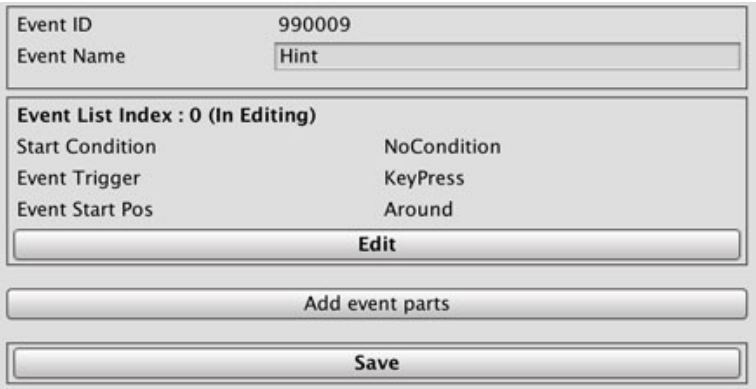
EventMasterData has parts groups called EventParts. EventParts defines an action in the dungeon, such as opening the door.

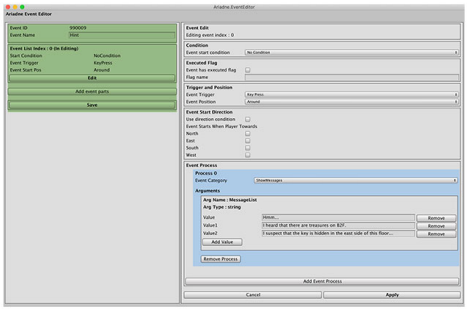
Each EventParts has a condition for starting an event. By checking the condition, Ariadne 3D Dungeon Maker decides the target of the processing event. You can set an "executed flag" to them, and you can use the flag to check the condition.

Ariadne 3D Dungeon Maker checks conditions by descending order from the end of the EventParts list.

EventMasterData Setting

Set data about the whole of the event file on the left side of the window.





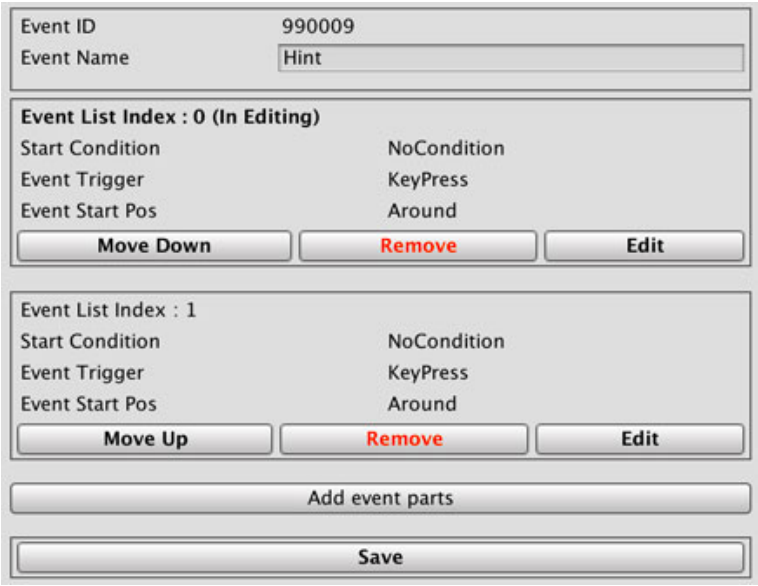
"**Event Name**" field defines the name of this event. This event name is shown in the MapEditor window.

To edit an EventParts, press [**Edit**] button. EventEditor shows settings of selected EventParts on the right pane.

If you want to add a new EventParts, press [**Add event parts**] button. This action adds a new element to the EventParts list.

To save EventMasterData, press [**Save**] button.

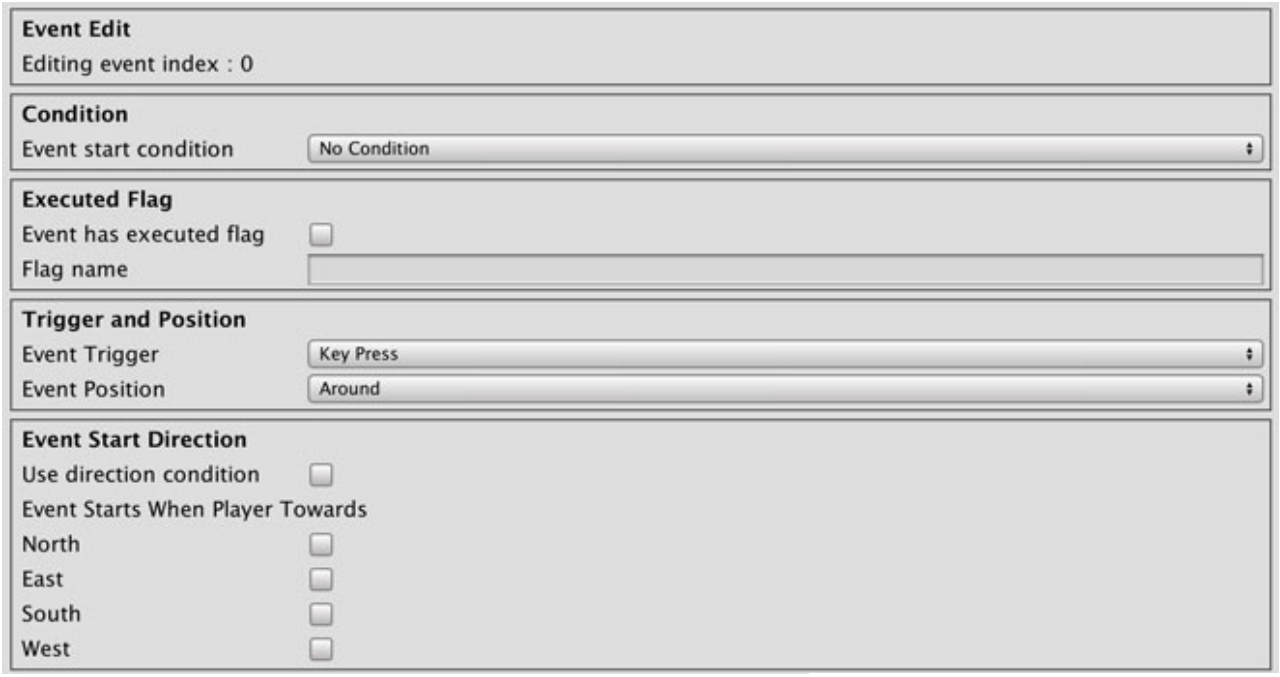
If there are more than one EventParts, EventEditor show [**Remove**] button on the left of [**Edit**] button. To remove the target EventParts, press [**Remove**] button.



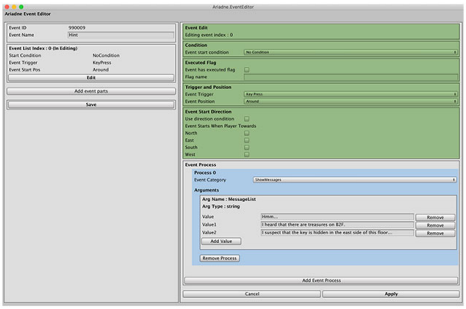
If you want to change the order of EventParts, click **[Move Down]** button or **[Move Up]** button.

EventParts Setting

Set conditions, flags, and triggers to decide which event should be executed.



After changing the EventParts settings, press [**Apply**] button to apply changes for the EventParts. If you want to revert changes, use [**Cancel**] button.



Condition

Select a condition for starting the EventParts.

Condition	Description
No Condition	The event starts without a condition.
Flag	The event starts when the selected flag is true.
Item	The event starts when the player has specified items. You can specify a comparison operator for comparing the value of the item.
Money	The event starts when the player has specified amount of money. You can specify a comparison operator for comparing the amount of money.

Executed flag

You can set an executed flag to EventParts. When you use this flag, check [**Event has executed flag**] and input the flag name. EventEditor uses this name in the selector of the condition setting.

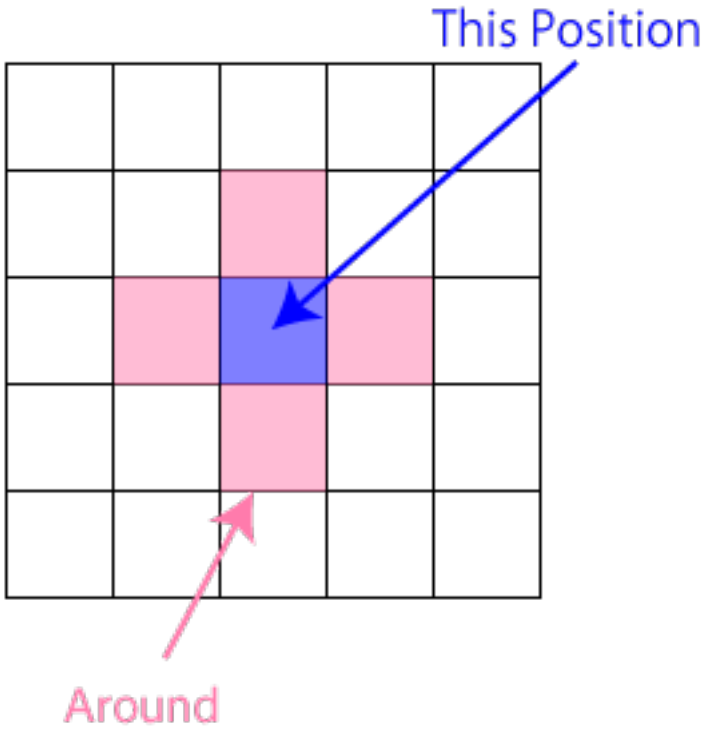
Trigger and Position

The event trigger defines how to start the event.

Trigger	Description
KeyPress	The event starts by pressing the space key.
Auto	The event starts after the player moving automatically.
On Entering Floor	The event starts when the player enters the floor.

The event position is the place where the event starts.

Position	Description
This Position	The event starts at the position where this event is defined.
Around	The event starts at the position next to this event.



Direction Condition

You can set conditions about the direction which the player object towards.

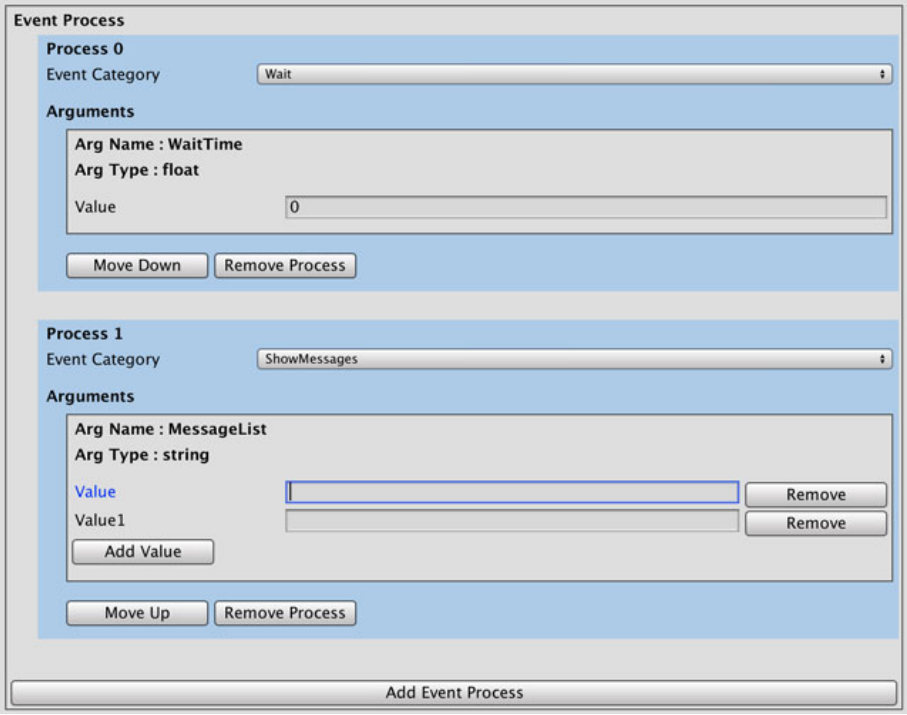
[Use direction condition] determines whether Ariadne uses conditions of directions or not.

If this check box is true, this event starts when the player object towards a specified direction.

If this check box is false, this event starts without regarding the direction of the player object.

Event process

Each EventParts holds processes as the list of EventProcess. EventProcessor processes from the beginning of this list.

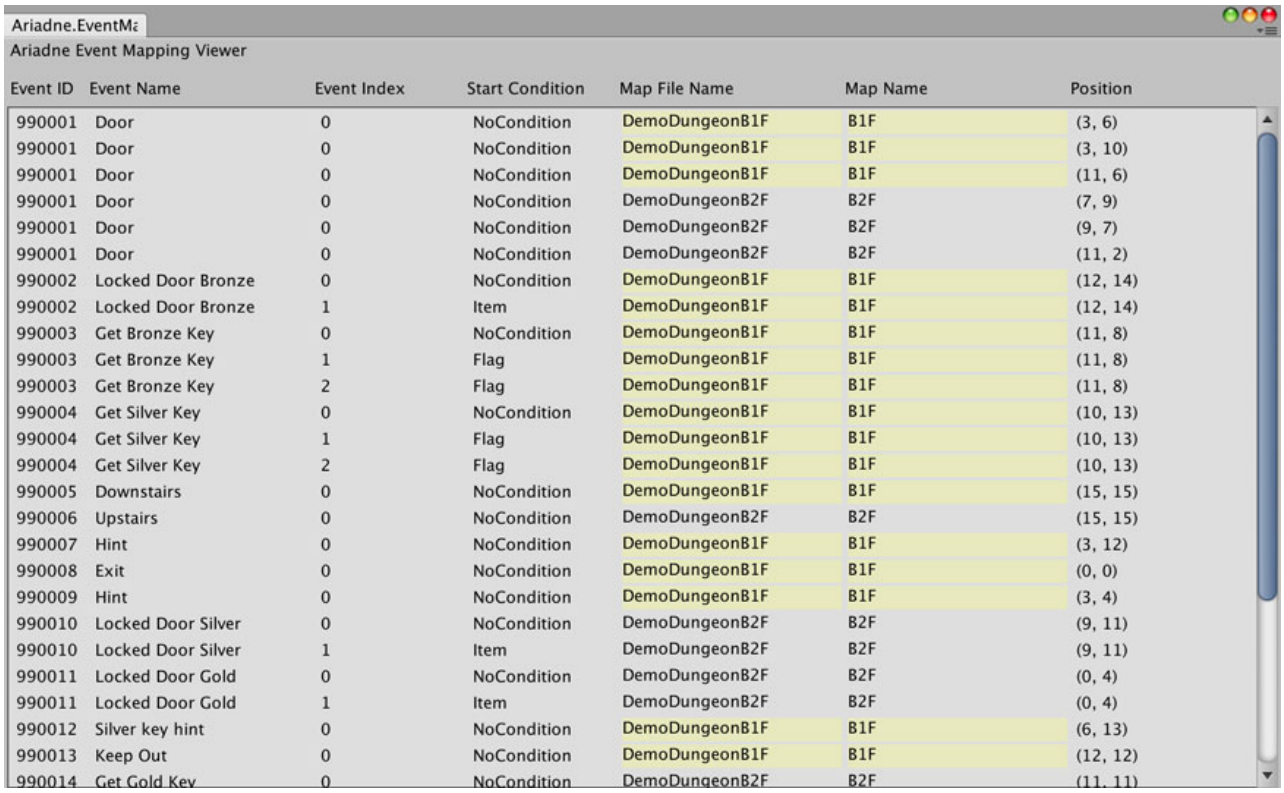


Each EventProcess has an EventCategory. If you set some arguments to the category in EventCategoryEditor, EventEditor shows fields of arguments. See also EventCategoryEditor.

For example, the category of “ShowMessages” has list field to show messages for some pages. **[Add Value]** button adds a list element, and **[Remove]** button removes the target element.

[Add Event Process] button adda an event process. You can change the order by using **[Move Down]** button and **[Move Up]** button. You can remove an event process to click **[Remove Process]** button.

EventMappingViewer



Event ID	Event Name	Event Index	Start Condition	Map File Name	Map Name	Position
990001	Door	0	NoCondition	DemoDungeonB1F	B1F	(3, 6)
990001	Door	0	NoCondition	DemoDungeonB1F	B1F	(3, 10)
990001	Door	0	NoCondition	DemoDungeonB1F	B1F	(11, 6)
990001	Door	0	NoCondition	DemoDungeonB2F	B2F	(7, 9)
990001	Door	0	NoCondition	DemoDungeonB2F	B2F	(9, 7)
990001	Door	0	NoCondition	DemoDungeonB2F	B2F	(11, 2)
990002	Locked Door Bronze	0	NoCondition	DemoDungeonB1F	B1F	(12, 14)
990002	Locked Door Bronze	1	Item	DemoDungeonB1F	B1F	(12, 14)
990003	Get Bronze Key	0	NoCondition	DemoDungeonB1F	B1F	(11, 8)
990003	Get Bronze Key	1	Flag	DemoDungeonB1F	B1F	(11, 8)
990003	Get Bronze Key	2	Flag	DemoDungeonB1F	B1F	(11, 8)
990004	Get Silver Key	0	NoCondition	DemoDungeonB1F	B1F	(10, 13)
990004	Get Silver Key	1	Flag	DemoDungeonB1F	B1F	(10, 13)
990004	Get Silver Key	2	Flag	DemoDungeonB1F	B1F	(10, 13)
990005	Downstairs	0	NoCondition	DemoDungeonB1F	B1F	(15, 15)
990006	Upstairs	0	NoCondition	DemoDungeonB2F	B2F	(15, 15)
990007	Hint	0	NoCondition	DemoDungeonB1F	B1F	(3, 12)
990008	Exit	0	NoCondition	DemoDungeonB1F	B1F	(0, 0)
990009	Hint	0	NoCondition	DemoDungeonB1F	B1F	(3, 4)
990010	Locked Door Silver	0	NoCondition	DemoDungeonB2F	B2F	(9, 11)
990010	Locked Door Silver	1	Item	DemoDungeonB2F	B2F	(9, 11)
990011	Locked Door Gold	0	NoCondition	DemoDungeonB2F	B2F	(0, 4)
990011	Locked Door Gold	1	Item	DemoDungeonB2F	B2F	(0, 4)
990012	Silver key hint	0	NoCondition	DemoDungeonB1F	B1F	(6, 13)
990013	Keep Out	0	NoCondition	DemoDungeonB1F	B1F	(12, 12)
990014	Get Gold Key	0	NoCondition	DemoDungeonB2F	B2F	(11, 11)

EventMappingViewer shows relations between FloorMapMasterData and EventMasterData. EventMasterData is related to some positions on the map, so this view shows them as a list.

How to open EventMappingViewer

Open EventMappingViewer by pressing [**Check Event Mapping**] button on the MapEditor window.

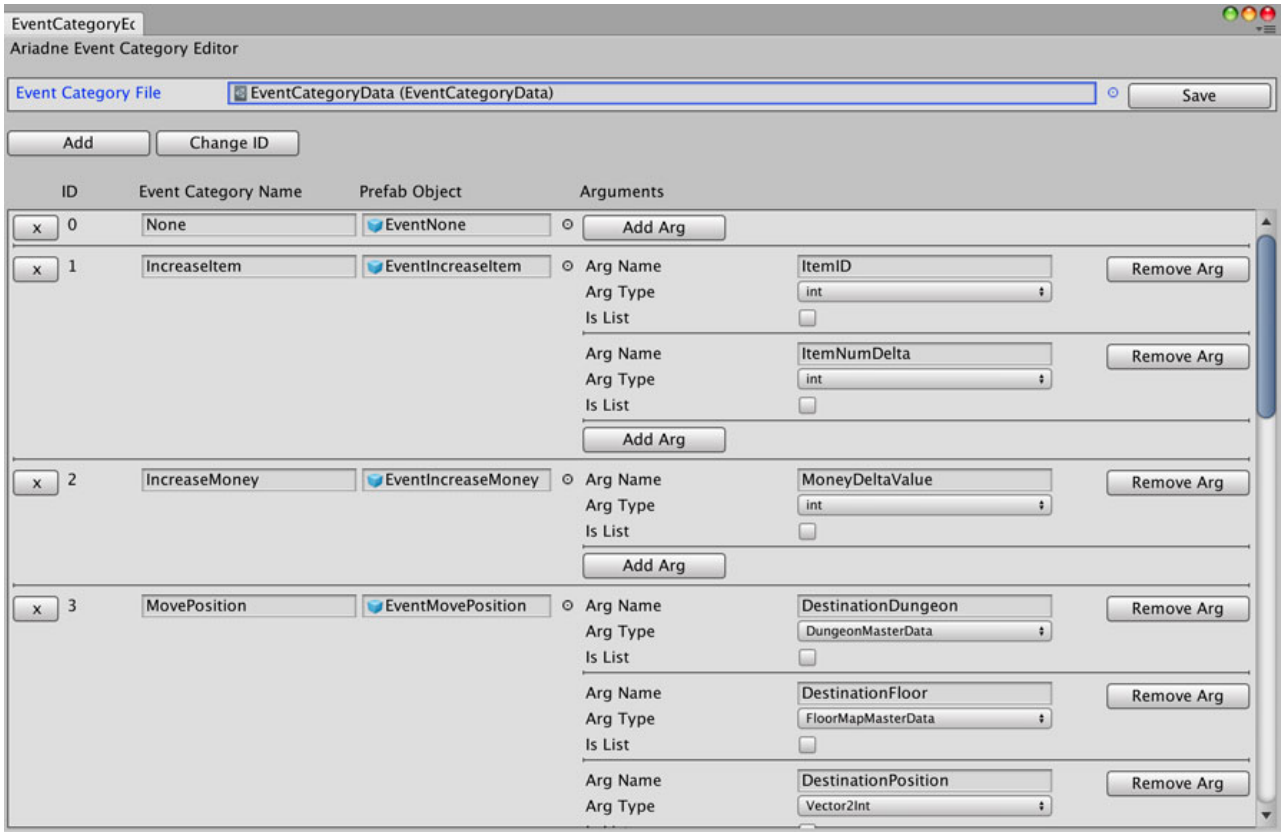
Viewer description

EventMappingViewer highlights events that related to the map editing in MapEditor.

Field	Description
Event ID	ID of events.
Event Name	Name of the event.
Event Index	Index of EventParts.
Start Condition	Condition to start the event.
Map File Name	Name of FloorMapMasterData file.
Map Name	Name of the floor.
Position	Position of the event.

How to close the window

To close this window, click the close button on the window. Or by clicking other windows, this window closes too.

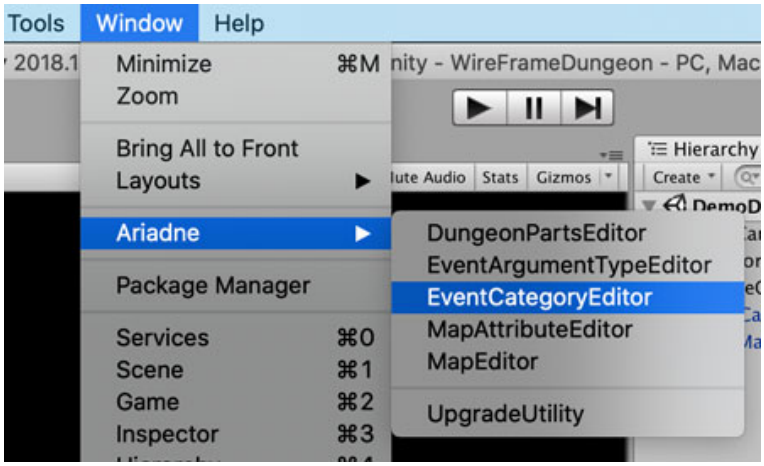


EventCategoryEditor

You can define categories of events in EventCategoryEditor. EventProcess uses these event categories.

How to open EventCategoryEditor

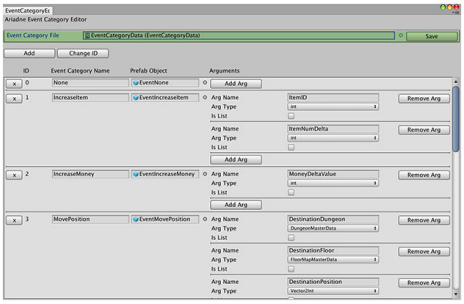
Open EventCategoryEditor from **[Window] -> [Ariadne] -> [EventCategoryEditor]** in the menu bar.



Settings pane

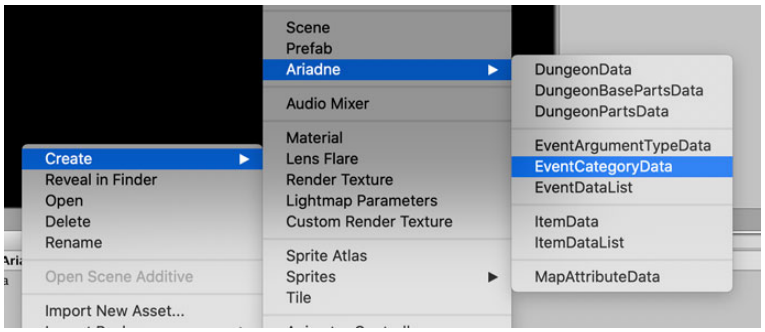
This section explains about setting panes in EventCategoryEditor.

File operation pane



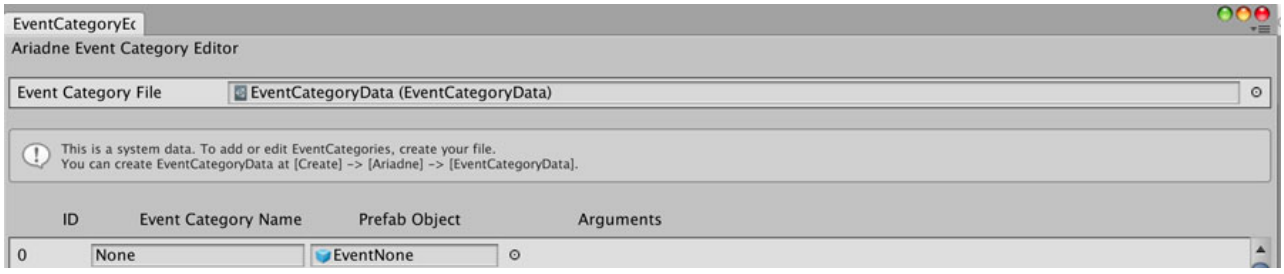
You can select the EventCategoryData file to edit in this pane.

You can create EventCategoryData from **[Create] -> [Ariadne] -> [EventCategoryData]** in the context menu.

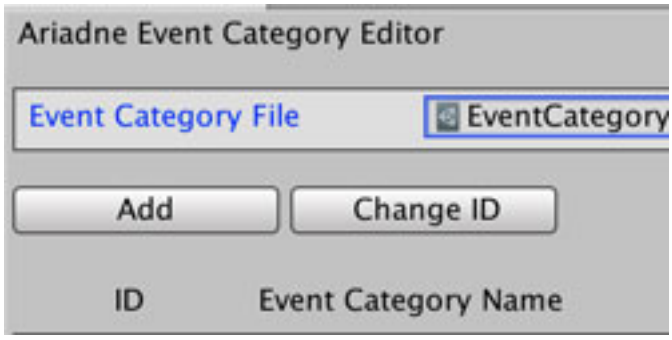


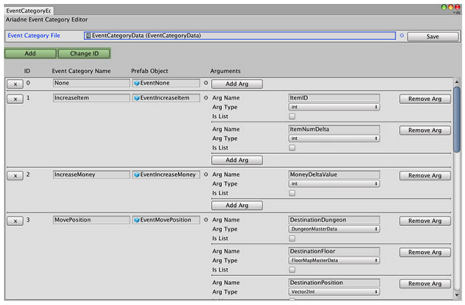
Click **[Save]** button to save after editing EventCategoryData.

If you select system data, EventCategoryEditor shows the following message and changes will not be applied.



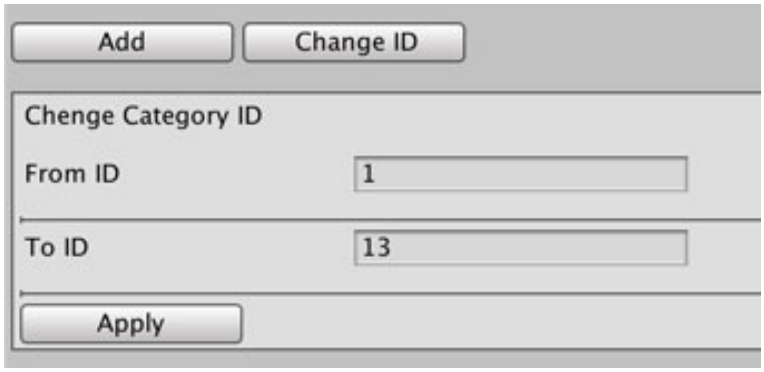
Button pane





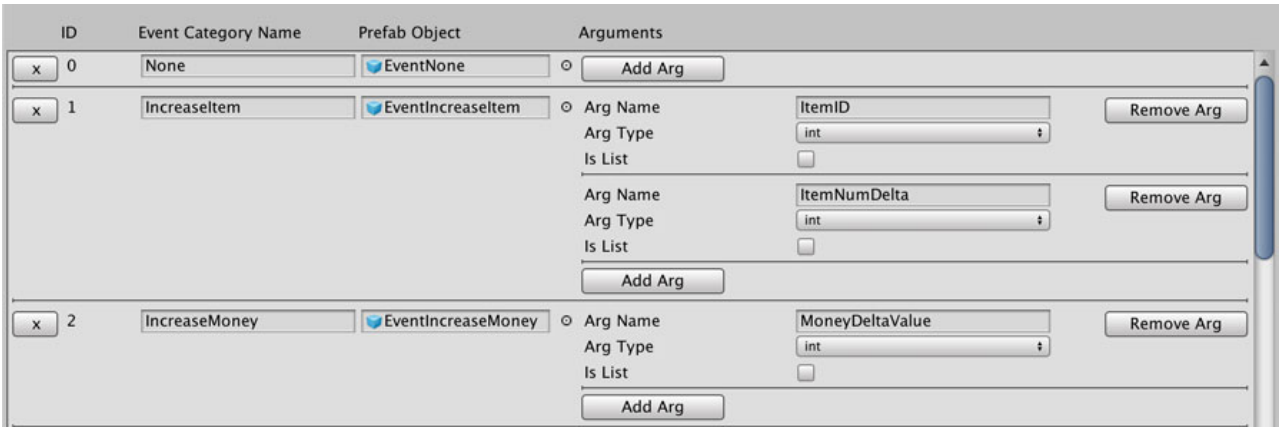
If you want to add a new category record, click **[Add]** button.

If you want to change the category ID, click **[Change ID]** button. EventCategoryEditor shows/hides changing fields.



EventCategoryEditor changes the event category ID from **“From ID”** to **“To ID”**. If IDs in these fields are valid, EventCategoryEditor shows **[Apply]** button. To apply to change, click this button. This change will be applied to all EventMasterData files in the project.

Category settings pane



You can define event categories in this pane.

[X] button on the left side of each record removes the record.

Input name of the category in **"Event Category Name"** field. EventEditor uses these names at the EventProcess setting.

Assign a prefab object which is attached to an event script to **"Prefab Object"** field. When you create an event script, inherit AriadneEventStrategyBase class, and implement the IAriadneEvent interface.

You can define arguments of the category in **"Arguments"** pane.

[Add Arg] button adds an argument. If you want to remove an argument, click **[Remove Arg]** on the right side of the record.

Input the name of the argument to **"Arg Name"** field. EventEditor uses this name.

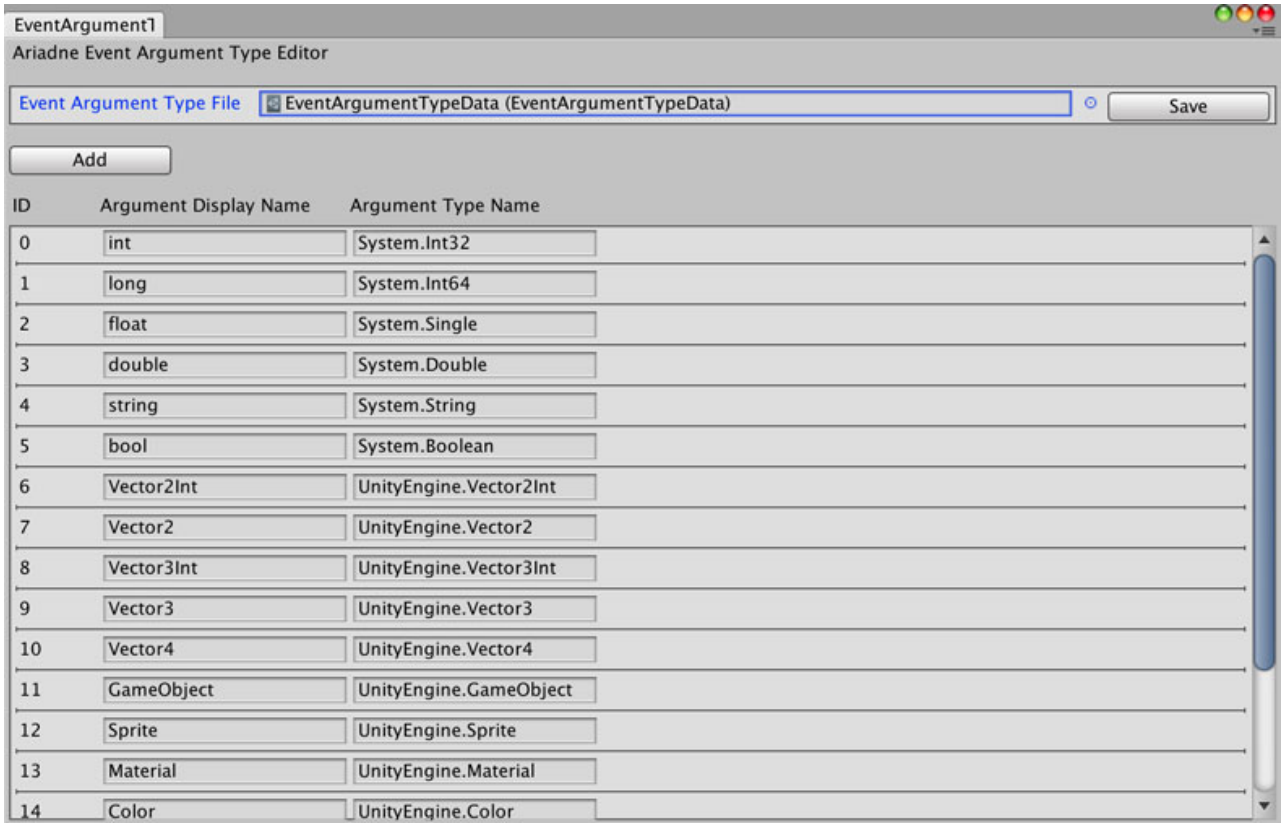
Specify the type of this argument at **"Arg Type"** field. This field shows types which are defined in EventArgumentTypeEditor.

Check the **"Is List"** check box if you want to input arguments as a list in EventProcess settings.

Event category in system data

System data has the following event categories.

ID	Category	Description
0	None	This category does nothing.
1	IncreaseItem	Increase the item specified ID.
2	IncreaseMoney	Increase player's money.
3	MovePosition	Move the player's position. You can move the player to another floor (such as stairs) or to the same floor like warping.
4	ExitDungeon	Exit process from the dungeon.
5	ShowMessages	Show the message window.
6	PlayAnimation	Play an animation.
7	MovePlayerToFront	Move the player to the front.
8	Wait	Wait seconds specified in args.

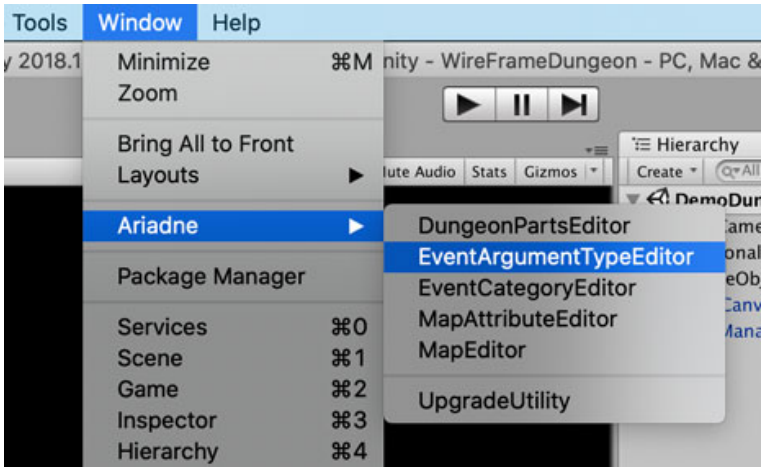


EventArgumentTypeEditor

You can define argument types of event categories in EventArgumentTypeEditor.

How to open EventArgumentTypeEditor

Open EventArgumentTypeEditor from **[Window] -> [Ariadne] -> [EventArgumentTypeEditor]** in the menu bar.

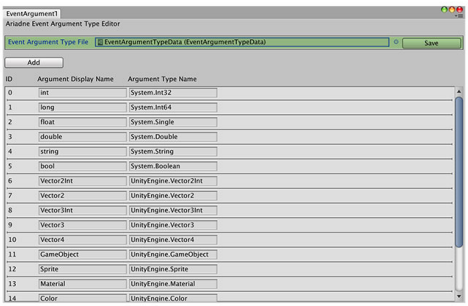


Settings pane

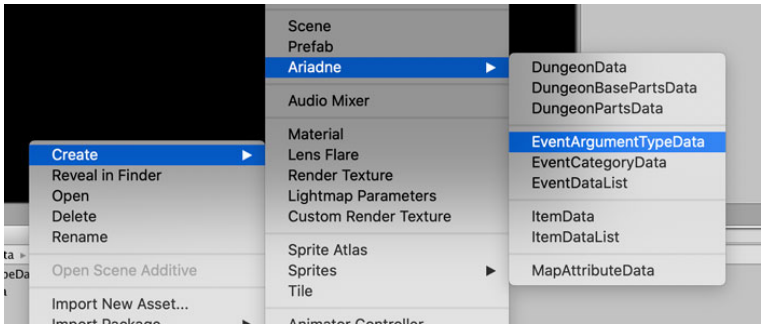
This section explains about setting panes in EventArgumentTypeEditor.

File operation pane

You can select the EventArgumentTypeData file to edit in this pane.

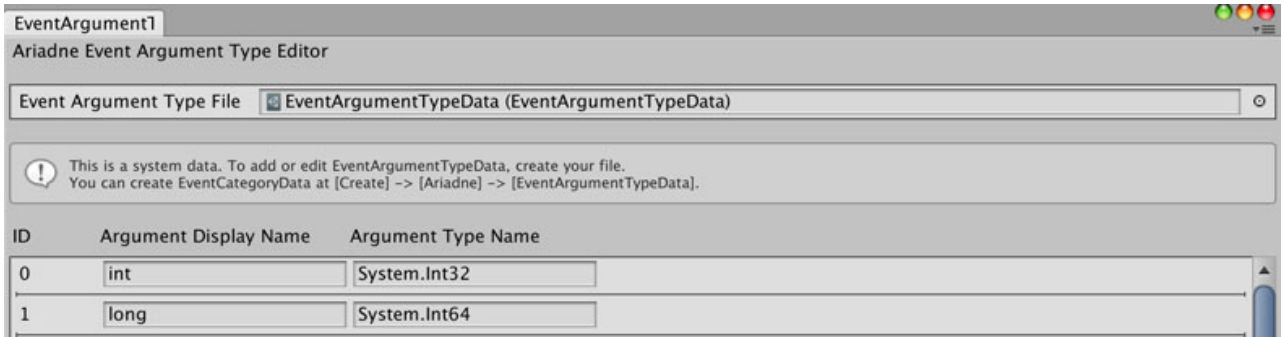


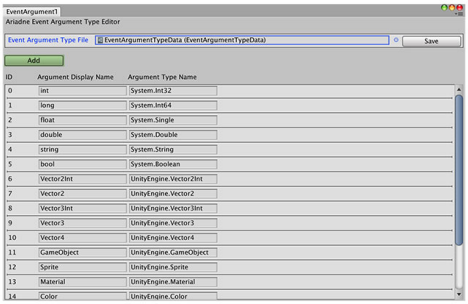
You can create EventArgumentTypeData from **[Create] -> [Ariadne] -> [EventArgumentTypeData]** in the context menu.



Click **[Save]** button to save after editing EventArgumentTypeData.

If you select system data, EventArgumentTypeEditor shows the following message and changes will not be applied.

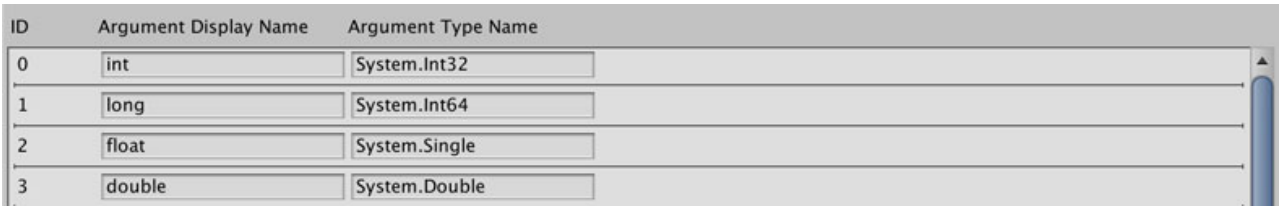




Button pane

If you want to add a new argument type record, click **[Add]** button.

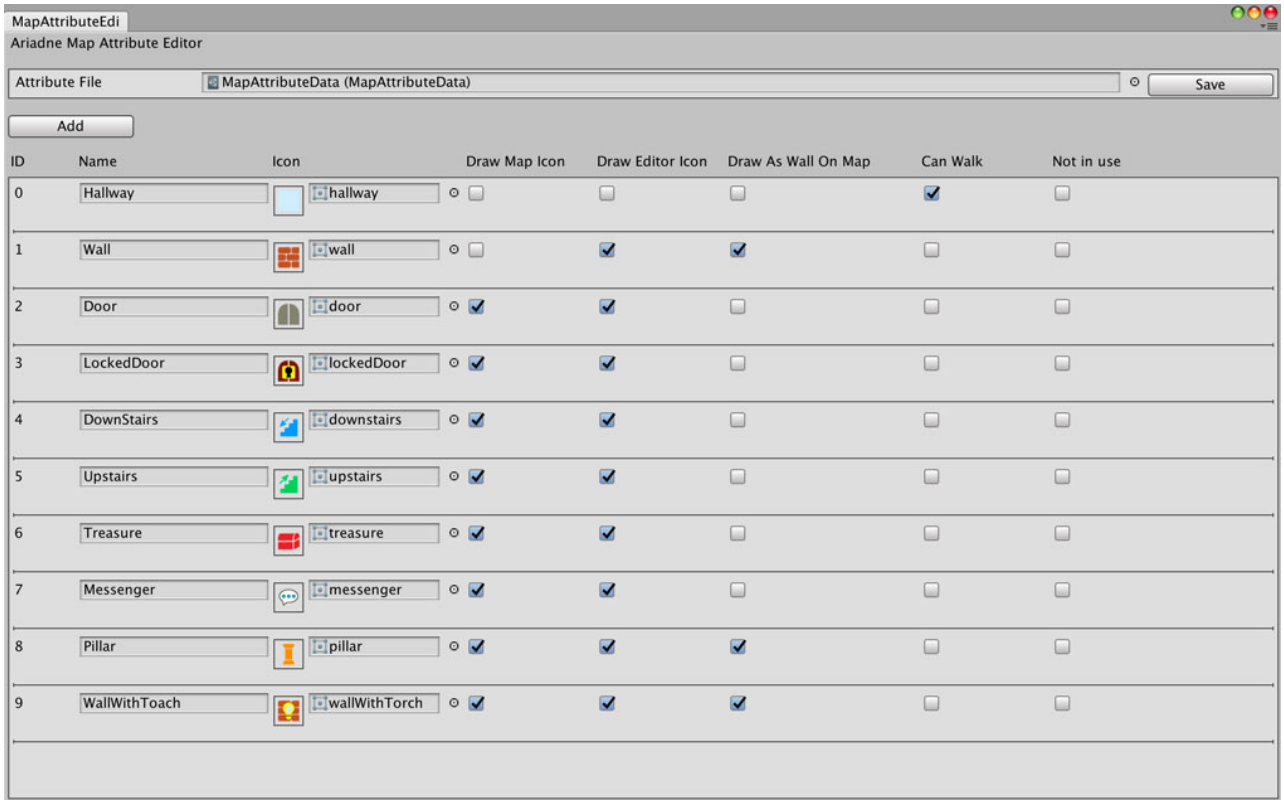
Argument type settings pane



EventArgumentTypeEditor assigns **“ID”** automatically.

EventCategoryEditor uses **“Argument Display Name”** in argument type selectors.

Input type name of the argument to **“Argument Type Name”** field. (Include namespace)

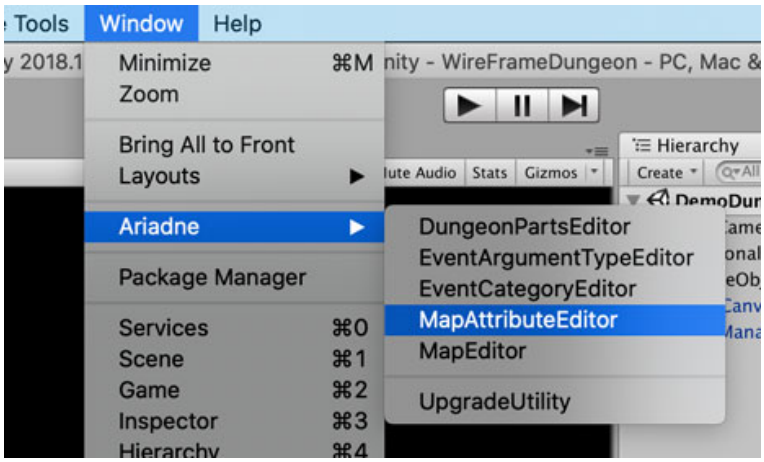


MapAttributeEditor

You can define map attributes in MapAttributeEditor.

How to open MapAttributeEditor

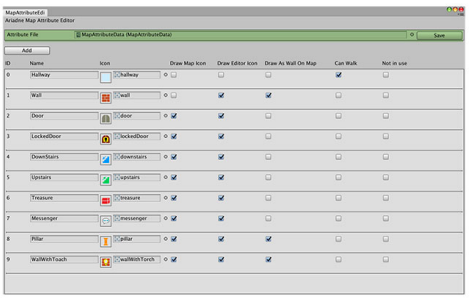
Open MapAttributeEditor from **[Window] -> [Ariadne] -> [MapAttributeEditor]** in the menu bar.



Settings pane

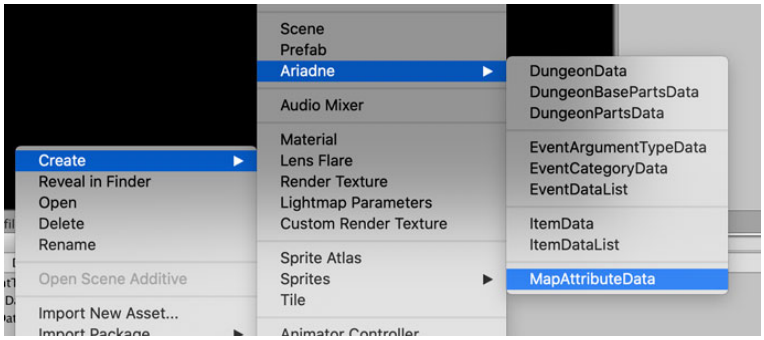
This section explains setting panes in MapAttributeEditor.

File operation pane

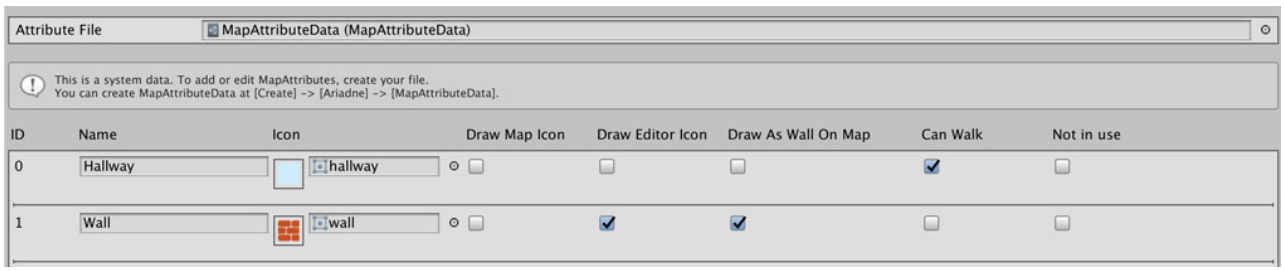


You can select the MapAttributeData file to edit in this pane.

You can create MapAttributeData from **[Create] -> [Ariadne] -> [MapAttributeData]** in the context menu.

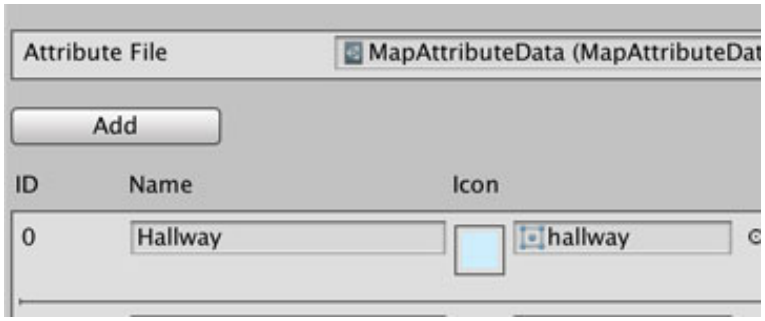


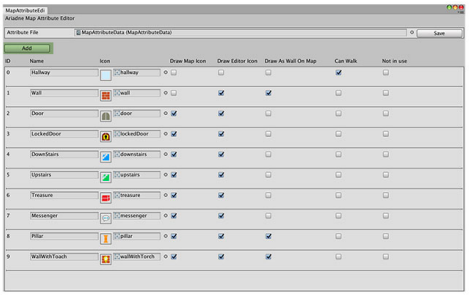
Click **[Save]** button to save after editing MapAttributeData.



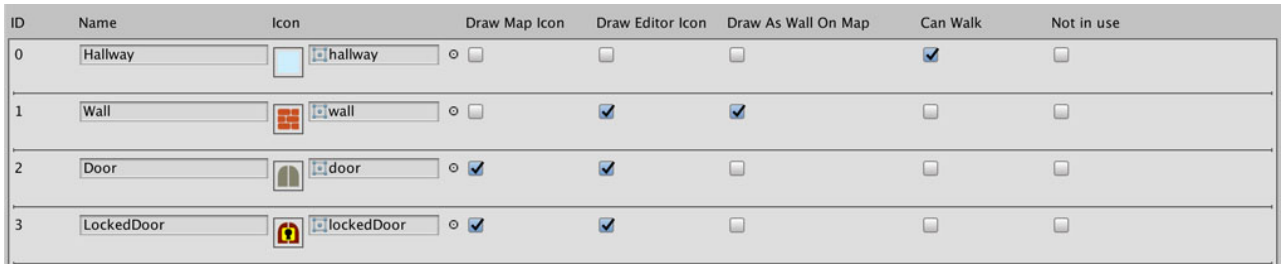
If you select system data, MapAttributeEditor shows the following message and changes will not be applied.

Button pane





If you want to add a new map attribute record, click **[Add]** button.



Map attribute settings pane

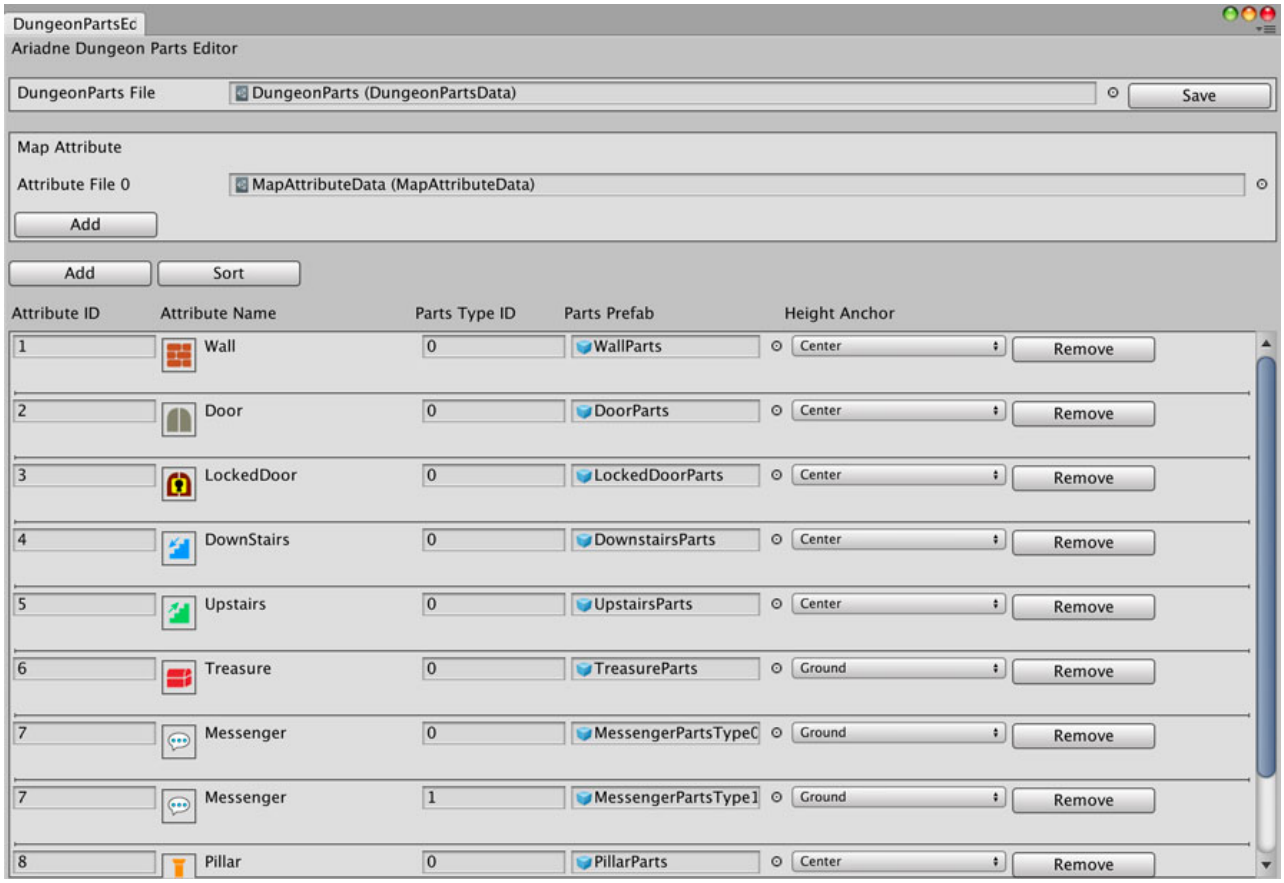
MapAttributeEditor assigns **“ID”** automatically.

Input an attribute name to the **“Name”** field. MapEditor uses this name to show information about the position on the map.

Assign a map icon to the **“Icon”** field. MapEditor uses this icon on the toolbar and map. Ariadne uses this icon in the uGUI map on runtime. It is recommended to set icon size 32 * 32.

You can set the following check box fields.

Field	Description
Draw Map Icon	Setting about showing this icon on the uGUI map at runtime. True: Show this icon. False: Don't show this icon.
Draw Editor Icon	Setting about showing this icon on the map at MapEditor. True: Show this icon. False: Don't show this icon.
Draw As Wall On Map	Setting about showing this icon as a wall (masked area) on the uGUI map at runtime. True: Show this icon as a wall. False: Don't show this icon as a wall.
Can Walk	Setting about whether the player can walk on this attribute or not. True: The player can walk on this map attribute. False: The player can't walk on this map attribute.
Not In Use	Setting about whether you use this attribute or not. If this check box is true, MapEditor doesn't show this icon on the toolbar. True: Don't use this attribute. False: Use this attribute.

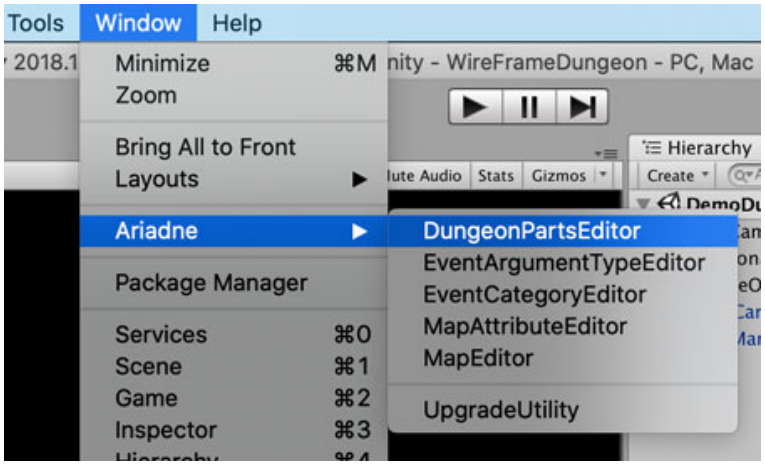


DungeonPartsEditor

You can define dungeon parts that correspond to map attributes in DungeonPartsEditor.

How to open DungeonPartsEditor

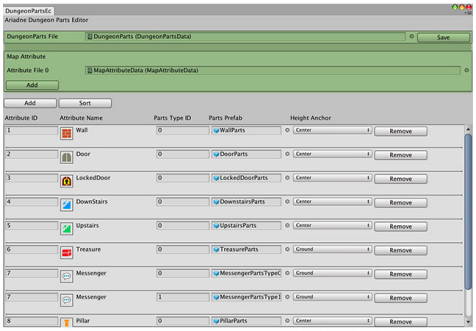
Open DungeonPartsEditor from **[Window] -> [Ariadne] -> [DungeonPartsEditor]** in the menu bar.



Settings pane

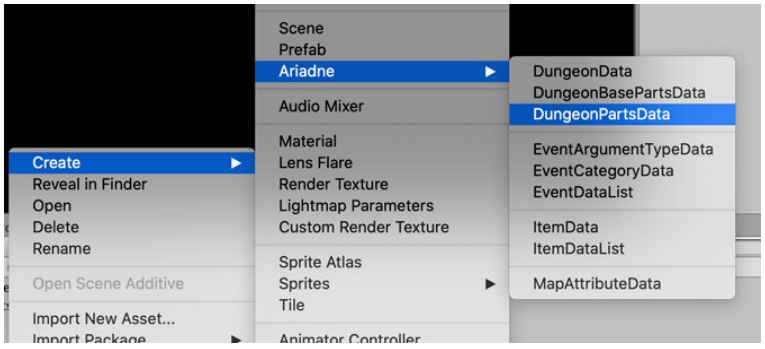
This section explains setting panes in DungeonPartsEditor.

File operation pane



You can select the DungeonPartsData file to edit in this pane. Set a DungeonPartsData to **“DungeonParts File”** to edit.

You can create DungeonPartsData from **[Create] -> [Ariadne] -> [DungeonPartsData]** in the context menu.

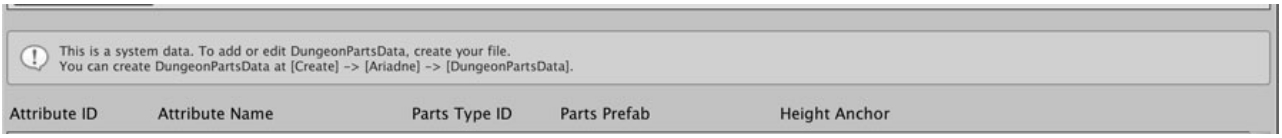


Click **[Save]** button to save after editing DungeonPartsData.

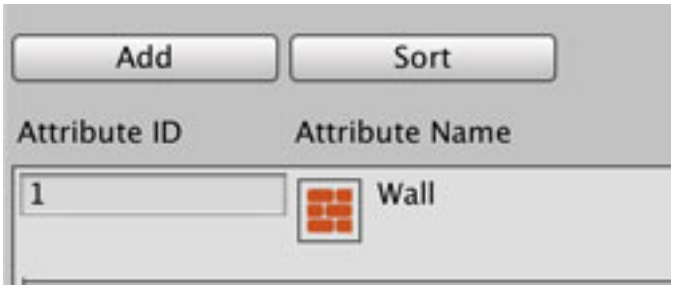
You can set some MapAttributeData to **“Map Attribute”** fields. DungeonPartsEditor doesn’t save this field, but use to show map icons on this editor. To add a field, click **[Add]** button.

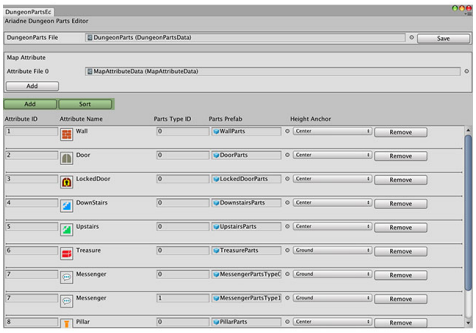
To remove a field, click **[Remove]** button on the right side.

If you select system data, DungeonPartsEditor shows the following message and changes will not be applied.



Button pane





If you want to add a new parts record, click **[Add]** button.

Click **[Sort]** button when you want to sort records. It sorts records by ascending of ID.

Dungeon parts settings pane

Input attribute ID to **“Attribute ID”** you want to correspond to the parts. If MapAttributeData includes the specified ID, DungeonPartsEditor shows the icon and name of the attribute in the **“Attribute Name”** field.

Input a parts type ID to the **“Parts Type ID”** field. You can set multiple parts to a map attribute. You can select these parts types in MapEditor.

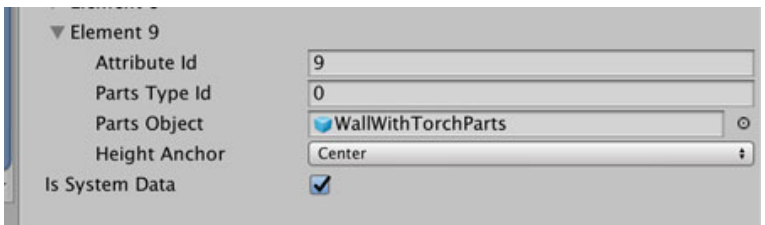
Assign a prefab object which you want to instantiate to the **“Parts Prefab”** field. You can select a height anchor of the object from the following anchor.

Anchor	Description
Center	Specify the height anchor as the center of the object.
Ground	Specify the height anchor as the ground of the object.
Ceiling	Specify the height anchor as the ceiling of the object.

About demo parts

Ariadne has some demo parts. “**WallWithTorchParts**” is the parts that are combined particle and light. If you build for WebGL1.0 (such as the environment of disabling the hardware acceleration), this part might cause shade error. For the environment, it is recommended not to use default DungeonPartsData or change **”WallWithTorchParts”** to other parts such as **“WallParts”**.

If you want to edit default DungeonPartsData, open DungeonPartsData in the Inspector window and uncheck the **“Is System Data”** field. After editing the data, check this field again.



Chapter 4

Data Files in Ariadne

This chapter explains about data files in Ariadne. Ariadne uses the following data files.

Data file	Description
DungeonMasterData	This file holds settings about the dungeon.
DungeonPartsData	This file holds dungeon parts to instantiate at run time.
DungeonBasePartsData	This file holds the parts to reference size, ceiling parts, and ground parts.
FloorMapMasterData	This file holds floor information in the dungeon.
MapAttributeData	This file holds definitions of map attributes.
EventMasterData	This file holds event information.
EventCategoryData	This file holds definitions of event categories.
EventArgumentTypeData	This file holds definitions of argument types of event categories.
EventDataList	This file holds a list of EventMasterData used in the game.
ItemMasterData	This file holds item information.
ItemDataList	This file holds a list of ItemMasterData used in the game.

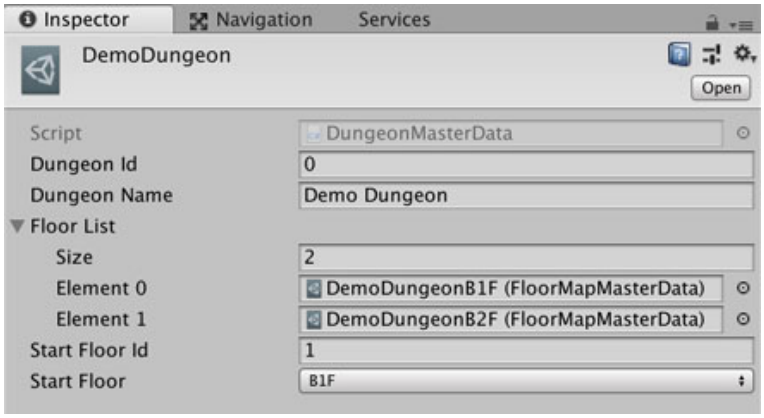
DungeonMasterData

DungeonMasterData is data that holds some FloorMapMasterData and ID of the start floor in the dungeon. After creating some FloorMapMasterData, you should create DungeonMasterData and set this to GameController.

How to create

Create DungeonMasterData from **[Create] -> [Ariadne] -> [DungeonData]** in the context menu.

Set FloorMapMasterData



Set FloorMapMasterData to DungeonMasterData via the Inspector window.

You can specify the "**Start Floor Id**" by using the selector.

Attention: If there are some FloorMapMasterData which have the same floor ID, the selector shows only one of them.

So you have to specify different IDs for those floors.

DungeonPartsData

DungeonPartsData is a set of dungeon parts prefabs. Scripts that draw dungeons instantiate them at runtime.

How to create

Create DungeonPartsData from **[Create] -> [Ariadne] -> [DungeonPartsData]** in the context menu.

Set dungeon parts prefabs

Default prefabs are placed at [**Assets/Ariadne/Prefabs/DungeonParts**] folder.

Use this data to assign to FloorMapMasterData. You can switch dungeon parts for each floor.

To set up parts that correspond to map attributes, use DungeonPartsEditor. See also DungeonPartsEditor.

DungeonBasePartsData

Ariadne uses DungeonBasePartsData to reference the size of the parts. This file also holds the outside wall, ground object, and ceiling object.

How to create

Create DungeonBasePartsData from **[Create] -> [Ariadne] -> [DungeonBasePartsData]** in the context menu.

Set dungeon parts prefabs

Default prefabs are placed at [**Assets/Ariadne/Prefabs/DungeonParts**] folder.

Dungeon drawer scripts calculate the unit size of the dungeon based on the size of the object in the “Size Base Obj” field.

When you change the size of the wall prefab, you should check the value of the transform of the Player object. See also Player object.

Field	Description
Size Base Obj	Base object to reference the size of the parts.
Outside Wall Obj	This object is used to instantiate the outside wall of the floor.
Ground Obj	Set a ground parts prefab of the floor.
Ceiling Obj	Set a ceiling parts prefab of the floor.

FloorMapMasterData

FloorMapMasterData holds definitions of each map. Dungeon parts are instantiated based on this data.

How to create

Create FloorMapMasterData from **MapEditor**. To avoid misconfiguration, editing this data directly via the Inspector window is not recommended.

Data structure

FloorMapMasterData

Field	Description
Floor ID	ID of the floor. Don't duplicate with other floors in the dungeon.
Floor Name	Name of the floor.
Dungeon Parts	(Deprecated)
Dungeon Base Parts Data	Parts data for size reference.
Dungeon Parts Data List	Parts data list of the floor.
Map Attribute Data List	The list of definition files of map attributes.
Floor Size Horizontal	The horizontal size of the floor.
Floor Size Vertical	The vertical size of the floor.
Entrance Pos	Entrance position of the floor. Ariadne uses this value when this floor is the first floor of the dungeon.
Entering Dir	Entering direction of the floor. Ariadne uses this value when this floor is the first floor of the dungeon.
Map Info	Settings of each position on the floor.

MapInfo

Field	Description
Event ID	ID of the event.
Map Attr	Map attribute of the position.
Object Front	The front direction of the object that corresponds to the map attribute.
Messenger Type	Messenger object to instantiate on the position.

MapAttributeData

Ariadne uses MapAttributeData to hold map attributes.

How to create

Create MapAttributeData from **[Create] -> [Ariadne] -> [MapAttributeData]** in the context menu.

Define map attributes

Use MapAttributeEditor to define map attributes. See also MapAttributeEditor.

EventMasterData

EventMasterData holds information about events.

How to create

Create EventMasterData from **EventEditor**. To avoid misconfiguration, editing this data directly via the Inspector window is not recommended.

Data structure

EventMasterData

Field	Description
Event ID	ID of the event.
Event Name	Name of the event.
Event Parts	List of EventParts.

For more information, see EventParts Setting.

EventCategoryData

Ariadne uses EventCategoryData to hold definitions of event categories.

How to create

Create EventCategoryData from **[Create] -> [Ariadne] -> [EventCategoryData]** in the context menu.

Define event categories

Use EventCategoryEditor to define event categories. See also EventCategoryEditor.

EventArgumentTypeData

Ariadne uses EventArgumentTypeData to hold definitions of argument types that are used in event categories.

How to create

Create EventArgumentTypeData from **[Create] -> [Ariadne] -> [EventArgumentTypeData]** in the context menu.

Define event argument types

Use EventArgumentTypeEditor to define argument types of event categories. See also EventArgumentTypeEditor.

EventDataList

Ariadne use EventDataList to hold EventMasterData in the project.

How to create

Create EventDataList from **[Create] -> [Ariadne] -> [EventDataList]** in the context menu.

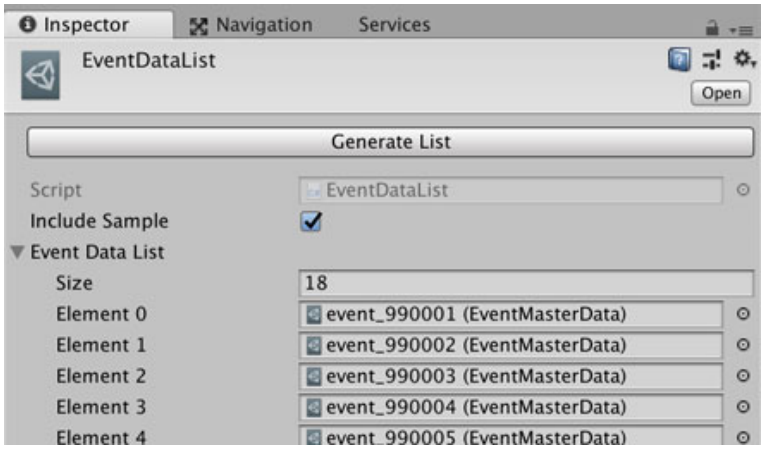
Set the list of EventMasterData

Set your event data to use in your game.

Ariadne can process events which included in this list. If you created an event data but this list doesn't include the data, Ariadne can't execute it.

EventDataList has **[Generate List]** button in inspector window. This button generates the list automatically. Ariadne retrieves all event data in the project and assigns them to this list.

Ariadne has some demo event data. If you don't want to include them, uncheck **[Include Sample]** and click **[Generate List]** button again to exclude sample data.



ItemMasterData

ItemMasterData stores an ID and name of the item. Ariadne doesn't manage other parameters such as a value of healing like a portion. To add these fields, you can create a partial class of ItemMasterData.

How to create

Create ItemMasterData from **[Create] -> [Ariadne] -> [ItemData]** in the context menu.

Data structure

ItemMasterData

Field	Description
Item Id	ID of the item.
Item Name	Name of the item.
Item Type	(Deprecated)
Door Key Type	(Deprecated)

Definition of item data

Ariadne manages item ID and item name. You can add your fields by using a partial class of ItemMasterData. Ariadne defines ItemMasterData as a partial class, so create a new script file and implement ItemMasterData as a partial class too.

ItemDataList

Ariadne uses ItemDataList to hold ItemMasterData in the project.

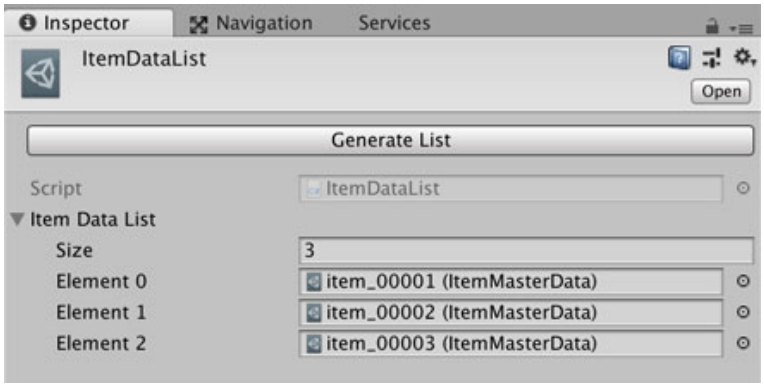
How to create

Create ItemDataList from **[Create] -> [Ariadne] -> [ItemDataList]** in the context menu.

Set the list of ItemMasterData

Set your item data in your game to this list. Ariadne retrieves item ID which you specified in EventProcess from this list.

ItemDataList has **[Generate List]** button in the Inspector window. This button generates the list automatically. Ariadne retrieves all item data in the project and assigns them to this list.



Chapter 5

Data Management

This chapter explains data management during runtime in Ariadne.

PlayerPosition

PlayerPosition is a static class to store information about the player position.

Managing data

Field	Description
playerPos	Current position of the player.
playerPosPre	Previous position of the player.
direction	Current direction of the player.
currentDungeonId	Dungeon ID that the player is exploring.
currentFloorId	Floor ID that the player is exploring.

ItemDataManager

ItemDataManager is a class to manage ItemData. ItemDataManager is attached to a scene object and it stores the dictionary of holding items.

Managing data

Field	Description
holdItemDict	The dictionary for item IDs and values.
money	Amount of money the player has.

EventFlagManager

EventFlagManager is a class to manage event flags. This class is attached to a scene object and it stores the dictionary of executed flags in EventParts.

Managing data

Field	Description
eventFlagDict	The dictionary for event executed flags and these states.

TraverseManager

TraverseManager is a static class to manage traverse data. The traverse data stores whether the player traversed the position or not. Ariadne identifies the traverse data by dungeon id, floor id, and the position on the floor.

The **“MapHall"** object in the scene uses traverse data to draw a map. See also MapHall.

Managing data

Field	Description
traverseList	The list of TraverseData.

TraverseData

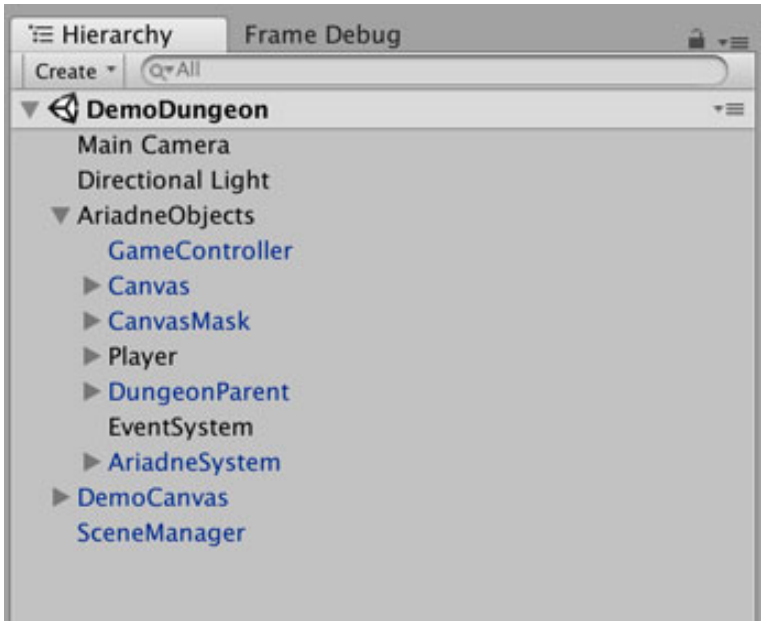
Field	Description
dungeonId	ID of the dungeon.
traverseDict	The dictionary of traverse data. The key string is generated by floor ID and the position.

Chapter 6

Scene Objects

This chapter explains GameObjects that Ariadne uses at runtime. Template files of scene object are placed in [**Ariadne/Prefabs/SceneObjects/Template**] folder as prefab.

The demo scene has the following objects. “AriadneObjects” is a parent object of objects which are used by Ariadne system.



The demo scene uses “DemoCanvas” and “SceneManager” object to control the scene.

You can override controller scripts by inheriting them.

GameController Object

This object has components of settings and references for system objects.

Components

DungeonSettings

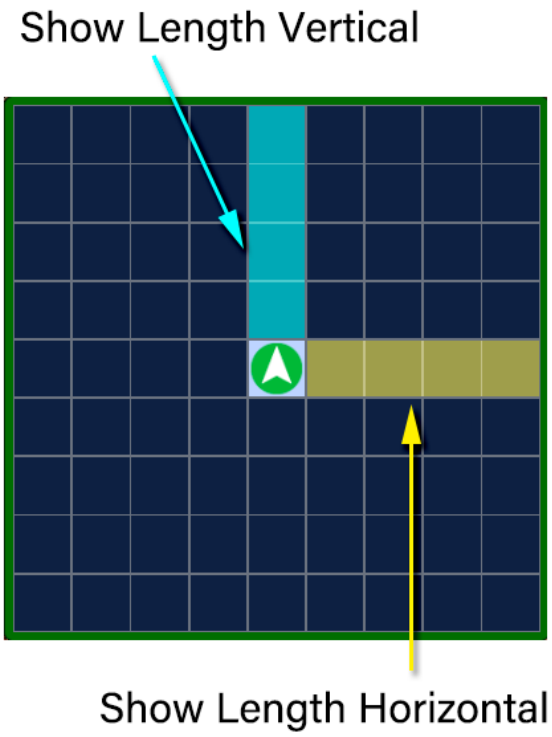
This component stores the setting for the dungeon.

Field	Description
Dungeon Data	Set the dungeon data that the scene uses.
Event Category Data List	Set EventCategoryData to process event in the scene.
Draw Outside Wall	The setting whether draw outside walls or not.
Outside Wall Size	Ariadne instantiates walls that are outside of the floor. The width of the walls is according to this value.

MapShowingSettings

This component stores setting of the uGUI map.

Field	Description
Show Length Horizontal	The number of the horizontal grid from the player's position to the end of the map.
Show Length Vertical	The number of the vertical grid from the player's position to the end of the map.
Smoothness	How many frames Ariadne spend when updates map uGUIs. As this value lower, updating becomes more smooth. But there is a possibility that it will be a performance cost.
Grid Line Width	The width of grid lines.
Enable Auto Mapping	Set whether to draw hallways based on TraverseData or not. If this setting is false, all hallways are visible regardless of



In this example, both "Show Length Horizontal" and "Show Length Vertical" are 4.

DrawManager

Manager script for drawing dungeons and the uGUI map. This script sends messages to scripts that instantiate dungeon parts.

Field	Description
Enable Draw Map	The setting whether draw uGUI map on the screen or not.

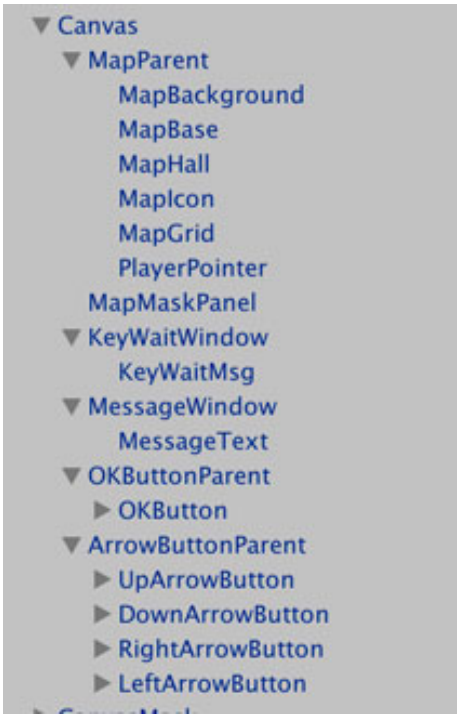
AriadneSceneSystemReference

This component holds references for Ariadne system object. System scripts in Ariadne use this references.

Field	Description
Event Processor Object	Set the object which has EventProcessor script.
Event Data Holder Object	Set the object which has EventDataHolder script.
Event Flag Manager Object	Set the object which has EventFlagManager script.
Item Data Holder Object	Set the object which has ItemDataHolder script.
Item Data Manager Object	Set the object which has ItemDataManager script.
Move Controller Object	Set the object which has MoveController script.

Canvas Object

“Canvas” object is a parent object of uGUI objects.



MapParent

Parent parts of uGUI map parts.

MapBackground

The background parts of the map. You can change the color of the background in the Image component.

MapBase

The base parts of the map. You can change the color of the base in the Image component.

MapHall

The hallway parts of the map. Ariadne draws the color of the hallway on this object. Specify the color in MapTraversedColor material.

MapIcon

Showing icon parts of the map. Ariadne draws icons on this object. Specify icons that correspond to map attributes. Default icons are placed on [**Assets/Ariadne/Images/Mapicon**] folder.

MapGrid

Gridlines parts of the map. Specify the grid color in MapGridColor material.

MapShowingSettings defines the width of the grid line.

PlayerPointer

The pointer icon of the player. You can change this icon at the Image component.

MapMaskPanel

The mask panel of the map on uGUI. In this demo scene, MapMaskPanel hides the map when the player is out of the dungeon.

KeyWaitWindow

Information window of waiting for the keypress. You can change the message of this window in the Text component of **KeyWaitMsg**.

MessageWindow

The message window for events. EventProcessor uses this window for showing messages.

OKButtonParent

The parent parts of the OK button. To change the label of this button, edit the Text component of **OKButtonLabel**.

ArrowButtonParent

The parent parts of the arrow buttons. To change the labels of these buttons, edit the Text component of each child object.

CanvasMask Object

CanvasMask object has a mask image object for the screen.

ScreenMaskPanel

The mask panel of the screen.

Player Object

The player objects that include a camera and lights. MoveController script controls this object by the input.

This object defines the height of the camera. When you change the sizes of prefabs, you should change **the y-value** of transform in the Player object.

The Player object has light objects for lighting the dungeon. By default, these objects have two lights (or light for mobile). Check your **"Pixel Light Count"** setting in Quality Settings.

DungeonParent Object

The parent parts of the dungeon parts.

GroundParent

The parent parts of the ground object of the dungeon.

WallParent

The parent parts of the wall objects of the dungeon.

CeilingParent

The parent parts of the ceiling object of the dungeon.

Field	Description
Draw Ceiling	The setting whether drawing the ceiling or not.

EventSystem Object

EventSystem object detects click event of uGUI. Unity editor creates this object when you create a new UI. **“AriadneObjects”** template includes canvas parts, so Ariadne needs this object.

AriadneSystem Object

AriadneSystem is a parent object of objects which have system scripts of Ariadne.

AriadneEventProcessor

AriadneEventProcessor object has EventProcessor script and AriadneEventStrategyFactory script. EventProcessor execute each event. AriadneEventStrategyFactory generates event object which is defined in EventCategoryEditor.

Field	Description
Debug Mode	Ariadne destroys instantiated event objects after 3 seconds the event has finished. If this check box is true, Ariadne doesn’t destroy these events.

AriadneEventDataHolder

AriadneEventDataHolder object has EventDataHolder script. This script holds a reference for EventDataList data.

Field	Description
Event List	Set an EventDataList file.

AriadneEventFlagManager

AriadneEventFlagManager object has EventFlagManager script. This script holds information about executed flags of events. If you want to save progress in your game, save the list of eventFlagDict field.

AriadneItemDataHolder

AriadneItemDataHolder object has ItemDataHolder script. This script holds a reference for ItemDataList data.

Field	Description
Item List	Set an ItemDataList file.

AriadneItemDataManager

AriadneItemDataHolder object has ItemDataHolder script. This script holds a dictionary of item num and player's money. You can get the item num dictionary from the **"holdItemDict"** field. And you can get player's money from **GetMoneyValue()** method. To keep the progress of your game, save these values.

AriadneMoveController

AriadneMoveController object has MoveController script. This script controls the player moving in the dungeon. MoveController checks events of each position. After moving, MoveController calls OnPostMoveEvent() method via IPostMoveNotify interface to customize the process at this timing.

To integrate with your system, see also Integration.

Field	Description
Move Wait	Set the time for moving. (Seconds)
Use UGUI Button	Setting whether to show the uGUI buttons to control the game or not. It is recommended to use this option when the build target has no control device such as mobiles.
Ok Button Parent	Set the reference for the parent object of the uGUI OK button.
Arrow Button Parent	Set the reference for the parent object of the uGUI arrow button.
Screen Mask Image	Set the reference for the screen mask image.
Map mask Image	Set the reference for the mask image of the uGUI map.
Map Fade Time	Set the time for fading the mask image of the uGUI map.
Key Wait Window	Set the reference for the key wait window image of the uGUI map.
Key Wait Fade Time	Set the time for fading the key wait window image of the uGUI map.

Chapter 7

Integration

This chapter explains integration with your system and Ariadne.

Enter & Exit the dungeon

Ariadne supports generating dungeons, controlling the movement of the player in dungeons, and executing events. To integrate Ariadne into your game, some settings are required.

To load the dungeon scene from scripts, add the scene to **"Scenes In Build"** in BuildSettings.

Then, create a script for scene management. In this script, implement the process to call OnEnterDungeon() method of MoveController. This method processes to instantiate the dungeon. Ariadne has a sample code for this process in **DemoSceneManager**.

In the DemoSceneManager script, Ariadne uses EventSystems of Unity and IEnterDungeon interface to execute processes of entering the dungeon at the Start() method.

To execute exit processes from the dungeon, use the ExitDungeon event. This event has an argument to get the name of the object which receives the notification of the exiting dungeon.

In the Demo scene, the ExitDungeon event has the name of the **“SceneManager”** object in its argument.

In detail, ExitDungeon notifies exit event to MoveController. And MoveController notifies this event to the receiver object. The receiver object should hold the script which implements the IExitDungeon interface. **DemoSceneManager** also includes this sample code.

You can inherit the MoveController script and can override methods.

Post Move Processes

Generally, 3D dungeon-crawling RPGs have a system of encountering enemies. And they also have a system of changing the player's status such as reducing the character's life who is poisoned.

To integrate these systems to Ariadne, use OnPostMoveEvent() method. In the Demo scene, Ariadne uses EventSystems of Unity and IPostMoveNotify interface to execute post processes by calling OnPostMoveEvent() method.

MoveController notifies this event to specified object. To set this object reference, call SetPostMoveEventObject() method from your script.

To receive the post-move notification, implement the IPostMoveNotify interface to the script in the receiver object.

After finished your processes, notify the finishing message to the MoveController script by using the IPostMoveProcessNotify interface. Then, the player can move.

See DemoScenePostMoveChecker script as sample code. Ariadne implements OnPostMoveEvent() method and callback method.

Ariadne implements method which sets the receiver object in **DemoSceneManager** script and call this method at Start() method:

moveController.SetPostMoveEventObject(gameObject);

Specify the reference of your receiver object like **"gameObject"** above the line.